

WYŻSZA SZKOŁA BANKOWA W GDAŃSKU
WYDZIAŁ EKONOMII I ZARZĄDZANIA W GDYNI

Adam Bielewicz
nr albumu 44084
Wojciech Woźniak
nr albumu 48582

Optymalizacja sieci neuronowej za pomocą algorytmów genetycznych

Praca inżynierska
na kierunku Informatyka

Praca napisana pod kierunkiem
dr Ireneusza Miciuły

Gdynia 2021

Spis treści

Wstęp	3
Rozdział 1: Algorytmy genetyczne.....	4
1.1 Algorytmy ewolucyjne wzorowane na biologicznej ewolucji	4
1.2 Podstawowe terminy dotyczące algorytmów genetycznych.....	7
1.3 Klasyczny algorytm genetyczny	8
Rozdział 2: Reprezentacja neuronów w matematyce i informatyce	14
2.1 Podstawowe informacje o budowie neuronu	14
2.2 Sztuczna reprezentacja neuronu za pomocą modeli matematycznych	17
2.3 Perceptron.....	20
Rozdział 3: Sieć neuronowa.....	23
3.1 Złożoność obliczeniowa i ograniczenia algorytmów	23
3.2 rozwinięcie algorytmów do struktury sieci neuronowej.....	24
3.3 Rodzaje funkcji aktywacji	25
3.4 Rodzaje i struktury sieci neuronowych	32
Rozdział 4: Metody optymalizacji i treningu sieci neuronowych	38
4.1. Optymalizacja	38
4.2. Metody uczenia i trenowania sieci neuronowych	38
4.3. Metoda gradientu prostego	42
4.3. Algorytm propagacji wstecznej	42
Rozdział 5: Przedstawienie stworzonego algorytmu genetycznego oraz analiza wyników optymalizacji.....	44
5.1. Opis działania algorytmu genetycznego.....	44
5.2. Optymalizowana sieć neuronowa	48
5.3. Wyniki optymalizacji	49
Zakończenie	55
Bibliografia	56
Spis rysunków	58
Spis tabel	58

Wstęp

Celem pracy jest ukazanie pozytywnych aspektów optymalizacji technologii sztucznej inteligencji za pomocą algorytmów genetycznych, w celu ustalenia optymalnej topologii sztucznej sieci neuronowej. Założenia obejmują przedstawienie algorytmu oraz wykazanie że możliwość prowadzenia równoległych obliczeń nad architekturą sztucznej inteligencji znacznie przyspiesza wydajność w stosunku do zastosowania tylko standardowego algorytmu propagacji zwrotnej. Będzie to również dowód że właściwy wybór parametrów drogą algorytmów genetycznych takich jak: rodzaj sztucznej sieci neuronowej, ilość neuronów i połączeń pomiędzy nimi ma kluczowe znaczenie dla skuteczności Sztucznej Inteligencji.

Jednym z głównych problemów związanych z projektowaniem sztucznych sieci neuronowych (SSN) jest odpowiednie dostosowanie struktury tychże. Przeprowadzono szereg badań porównujących wydajność uczenia sztucznych sieci neuronowych opartych o algorytmy ewolucyjne i opartych na gradiencie. Jednak wyniki badań nie były jednoznaczne, czasami nawet sprzeczne ze sobą, chociaż wykorzystano ten sam model i standardowy zestaw danych. Motywowane tym odkryciem, głównym celem tej pracy jest dokonanie kolejnego porównania między wariacjami metod optymalizacji takich jak Gradient a treningiem SSN opartym na algorytmie genetycznym ze szczególnym naciskiem na opracowany algorytm i metodologię porównawczą. Poza tym badany jest również wpływ operacji krzyżowania na trening Algorytmu Genetycznego.

Rozdział 1

Algorytmy genetyczne

1.1 Algorytmy ewolucyjne wzorowane na biologicznej ewolucji

Algorytmy ewolucyjne są zbiorem technik optymalizacji i poszukiwania rozwiązań zagadnień o wysokiej złożoności. Należą one do podzbioru obliczeń ewolucyjnych i należą do zbioru metod wyszukiwania opartych o rozwiązania heurystyczne. Ze względu na charakter pozwalający na dużą elastyczność zastosowania i łatwość w implementacji niezależnie od problemu stały się one skutecznym sposobem rozwiązywania problemów szeroko stosowanych w problemach optymalizacji. Mogą być z powodzeniem stosowane w wielu aplikacjach o dużej złożoności. W inteligencji obliczeniowej algorytm ewolucyjny jest podzbiorem obliczeń ewolucyjnych, należącym do grupy metaheurystycznych algorytmów optymalizacji opartym na populacji. Algorytm ewolucyjny wzoruje się na procesach ewolucji i ich odpowiednikom w biologii, takich jak technika kodowania (gen, chromosom), operatory genetyczne (mutacja, rekombinacja), funkcja celu (środowisko), selekcji rodziców (reprodukcja) i wiedzy o problemie który definiuje dobór warunku zakończenia (wiedza, doświadczenie, praktyka). Rozwiązania problemu optymalizacyjnego pełnią rolę jednostek w populacji, a wcześniej założona funkcja przystosowania determinuje jakość rozwiązań. Ewolucja populacji ma wtedy miejsce po ponownym zastosowaniu powyższych operatorów. Klasyczne metody optymalizacji różnią się od algorytmów genetycznych takimi cechami jak :

- Nie przetwarzają bezpośrednio parametrów funkcji opisującej problem, posługują się ich zakodowaną postacią.
- Nie wychodzą z pojedynczego punktu poszukiwań rozwiązania, wykorzystują pewną ich liczną populację.
- Nie korzystają z pochodnych ani innych tradycyjnych metod optymalizacji funkcji, robią to poprzez funkcję celu/przystosowania.

- Optymalny wynik nie należy do dziedziny rozwiązań deterministycznych, opiera się on o reguły probabilistyczne.
- Zbiór powyższych cech daje algorytmom genetycznym przewagę nad innymi metodami poszukiwań oraz odporność na problemy tradycyjnej optymalizacji.

Algorytmy ewolucyjne dobrze radzą sobie w odnajdywaniu przybliżonego rozwiązania wszystkich typów problemów, ponieważ w idealnym przypadku nie przyjmują żadnych założeń dotyczących podstawowej charakterystyki optymalnego rozwiązania. Elementy technik z algorytmów ewolucyjnych znajdują zastosowanie do modelowania ewolucji biologicznej są ograniczone do przeszukiwania procesów mikroewolucyjnych wzorowanych na procesach jakie zaobserwowano w komórkach. W większości rzeczywistych zastosowań algorytmów ewolucyjnych złożoność obliczeniowa jest czynnikiem zabraniającym. W rzeczywistości ta złożoność obliczeniowa wynika z oceny funkcji dopasowania. Przybliżenie sprawności jest jednym z rozwiązań pozwalających przezwyciężyć tę trudność. Jednak stosując pozornie proste algorytmy genetyczne może rozwiązać problemy o wysoki stopniu złożoności, dlatego może nie być bezpośredniego związku między złożonością algorytmu a złożonością problemu.

Podobne techniki różnią się reprezentacją genetyczną i innymi szczegółami implementacji, a także charakterem danego problemu.

Algorytm genetyczny - Poszukuje się rozwiązania problemu w postaci sekwencji liczb binarnych, stosując operatory takie jak rekombinacja i mutacja . Ten typ algorytmu jest często stosowany w problemach optymalizacyjnych.

Programowanie genetyczne - rozwiązaniami są programy, a ich przydatność jest określona przez ich zdolność do rozwiązania problemu obliczeniowego.

Programowanie ewolucyjne - podobne do programowania genetycznego, ale struktura programu jest stała, a jego parametry numeryczne mogą ewoluować wraz z kolejnymi krokami algorytmu.

Programowanie ekspresji genów - Podobnie jak programowanie genetyczne, ten algorytm również ewoluuje programy komputerowe, ale eksploruje układ genotyp-fenotyp, gdzie programy

komputerowe o różnych rozmiarach są zakodowane w liniowych chromosomach o stałej długości.

Strategia ewolucji - Pracuje z wektorami liczb rzeczywistych jako reprezentacjami rozwiązań i zazwyczaj używa samoadaptacyjnych współczynników mutacji.

Ewolucja różnicowa - bazuje na różnicach wektorowych i dlatego nadaje się głównie do numerycznych problemów optymalizacyjnych.

Neuroewolucja - podobna do programowania genetycznego, ale genomy reprezentują sztuczne sieci neuronowe poprzez opis struktury i wag połączeń. Kodowanie genomu może być bezpośrednie lub pośrednie.

System uczących się klasyfikatorów - tutaj rozwiązaniem jest zbiór klasyfikatorów należących do dziedziny reguł lub warunków. Początkowo, klasyfikatory były tylko binarne, ale obecnie zawierają typy rzeczywiste, sieci neuronowe.

Porównanie do procesów biologicznych

Pewnym ograniczeniem wielu algorytmów ewolucyjnych jest niemożliwość rozróżnienia między genotypem a fenotypem. W naturze, zapłodniona komórka jajowa przechodzi złożony proces zwany embriogenezą, aby stać się dojrzałym fenotypem. Uważa się, że takie pośrednie kodowanie czyni wyszukiwanie genetyczne bardziej odpornym, a także może poprawić ewolucyjność organizmu. Takie pośrednie kodowanie umożliwia również ewolucji wykorzystanie regularności w środowisku. Ostatnie prace w dziedzinie sztucznej embriogenezy, lub sztucznych systemów rozwojowych, starają się rozwiązać te problemy. A programowanie ekspresji genów z powodzeniem bada system genotyp-fenotyp, gdzie genotyp składa się z liniowych wielogenowych chromosomów o stałej długości, a fenotyp składa się z wielu drzew ekspresji lub programów komputerowych o różnych rozmiarach i kształtach.

1.2 Podstawowe terminy dotyczące algorytmów genetycznych

Inspiracją do powstania badań dotyczących algorytmów genetycznych było naśladowanie procesów zachodzących w naturze. Są one oparte na mechanizmach doboru naturalnego oraz dziedziczności. Każdy żyjący organizm posiada materiał genetyczny który warunkuje jego cechy, jest też w stanie go przekazywać nowym pokoleniom w procesie reprodukcji. Proces ten polega na wytworzeniu nowych organizmów poprzez krzyżowanie się genów jego rodziców. Często podczas rozmnażania dochodzi także do mutacji, czyli zamienienia pojedynczych genów. Organizmy posiadające korzystne cechy mają większą szansę na przeżycie i rozmnażanie, dzięki czemu korzystnie wpływające na przeżycie genów mają większą szansę na wystąpienie w populacji. Organizmom źle przystosowanym do środowiska ciężiej będzie przeżyć, a zatem i rozmnażać się.¹

Algorytmy genetyczne korzystają z terminologii zapożyczanej z genetyki. Podstawowymi pojęciami występującymi podczas ich opisywania jest populacja osobników, gen, chromosom, genotyp, fenotyp, allel. Używa się także odpowiadającym im technicznych określeń takich jak ciąg binarny czy struktura. Nazwy te używane są naprzemiennie.

- Populacja – zbiór organizmów o określonej liczebności
- Osobniki – inaczej Organizmy są to zakodowane w postaci chromosomów zbiory parametrów rozwiązania.
- Chromosomy – łańcuchy bądź ciągi kodowe w których zawierają się uporządkowane logicznie ciągi genów.
- Gen – inaczej cecha lub detektor – to pojedynczy element chromosomu, a co za tym idzie też genotypu
- Genotyp - struktura organizmu, zespół składających się na niego chromosomów. Osobnikiem populacji mogą być zarówno genotypy, jak i pojedyncze chromosomy, w przypadku gdy na genotyp tego organizmu składa się pojedynczy chromosom.
- Fenotyp – zestaw wartości odpowiadających genotypowi, zdekodowana struktura, zbiór parametrów rozwiązania.
- Allel – inaczej wartość cechy, wariant cechy - wartość danego genu

¹ Rutkowski L., *Metody i techniki sztucznej inteligencji*, Wydawnictwo naukowe PWN, Warszawa 2012, s. 237

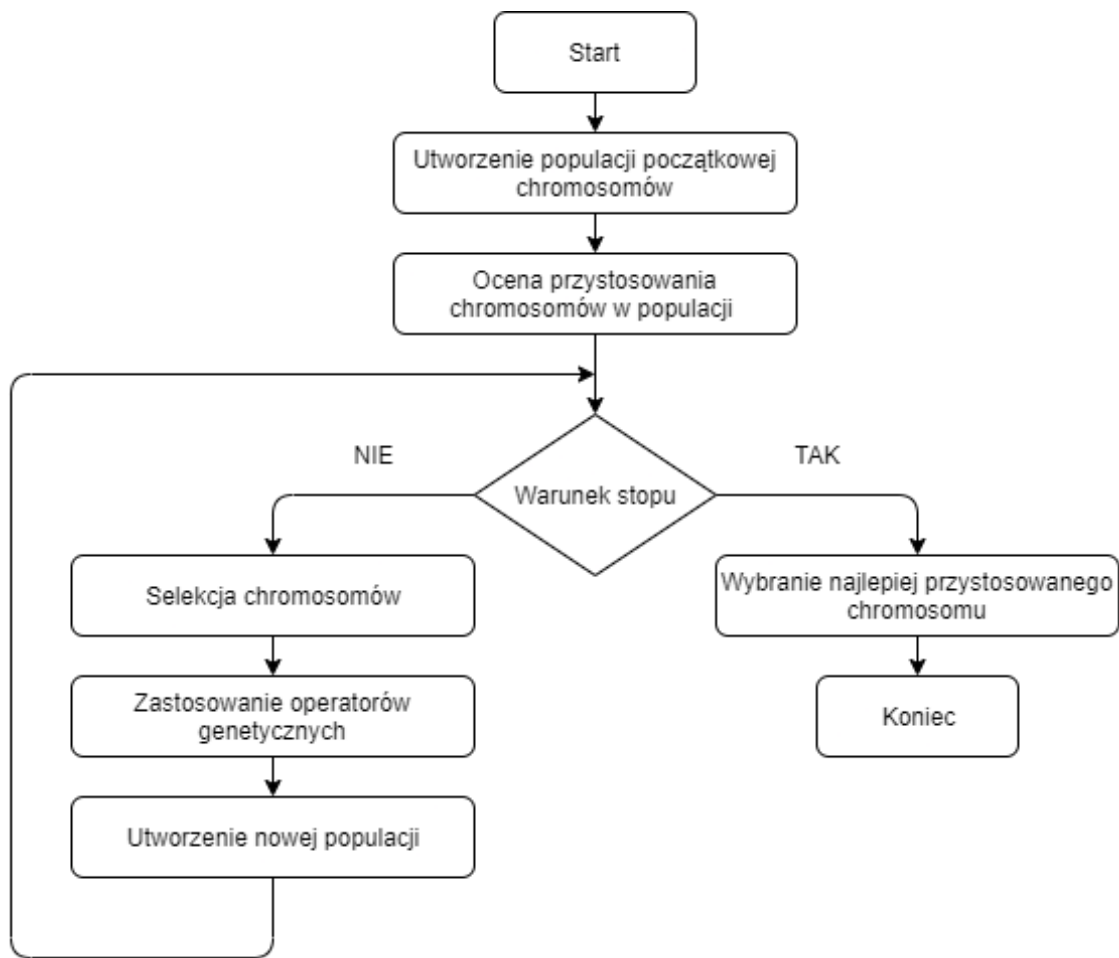
- Locus – pozycja wskazująca miejsce danego genu w chromosomie.
- Pokolenie – inaczej generacja – jest to każda kolejna iteracja algorytmu, w której występuje nowa populacja osobników.

Ważnym terminem przy omawianiu algorytmów genetycznych jest także funkcja przystosowania, nazywana też funkcją oceny. Stanowi ona miarę przystosowania danego organizmu w populacji. Funkcja ta wywodzi się również bezpośrednio z genetyki w której pozwala ocenić stopień przystosowania poszczególnych osobników w populacji. Na jej podstawie możemy ocenić jakie osobniki są, zgodnie z ewolucyjną teorią, najsilniejsze. Takie osobniki będą miały największą szansę na przekazanie swoich genów kolejnym pokoleniom.²

1.3 Klasyczny algorytm genetyczny

W tym podrozdziale przedstawiony zostanie schemat blokowy podstawowego algorytmu genetycznego oraz szczegółowy opis poszczególnych bloków. Każdy z etapów zostanie zobrazowany poprzez działania na grupie czterech organizmów składających się z sześciobitowych chromosomów.

² Rutkowski L., op. cit., s. 239-240



Rysunek 1 - Schemat blokowy algorytmu genetycznego

Źródło: Opracowanie własne na podstawie książki Rutkowski L., Metody i techniki sztucznej inteligencji

Utworzenie populacji początkowej chromosomów to najczęściej przeprowadzane w sposób losowy wygenerowanie pewnej liczby osobników reprezentowanych poprzez ciągi binarne o określonej długości. W przykładzie wygenerowane są cztery osobniki które posiadają po jednym chromosomie na który składa się sześć genów:

110010

010001

001011

101001

Ocena przystosowania chromosomów w populacji polega na ustaleniu funkcji przystosowania dla każdego z osobników. Postać funkcji zależy od tego jaki problem ma rozwiązywać algorytm genetyczny. Im większa wartość funkcji tym lepiej przystosowany do przeżycia jest dany organizm i tym większe jego szanse na reprodukcję. W przykładzie algorytm będzie poszukiwał maksimum funkcji $f(x) = x^3/2$, a więc przystosowanie będzie wynikiem podniesienia decymalnej wartości ciągu binarnego (fenotypu) do potęgi $3/2$. Maksimum takiej funkcji występuje dla 111111 i wynosi 500. Ocena przystosowania poszczególnych osobników prezentuje się następująco:

numer	ciąg kodowy	Fenotyp genu	Przystosowanie	% całości
1	110010	50	354	48.9%
2	010001	17	70	8.1%
3	001011	11	36	4.6%
4	101001	41	263	34.4%

Tabela 1 - Wartości przystosowania dla przykładowych organizmów

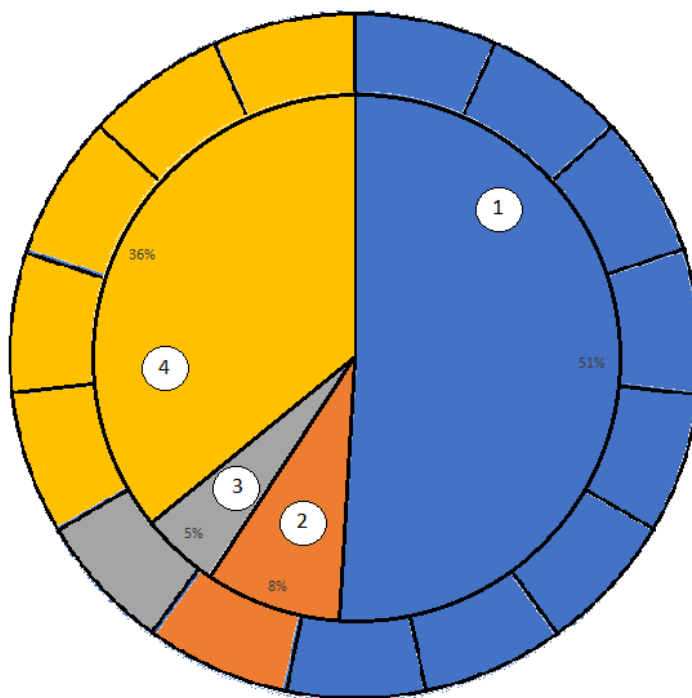
Źródło: Opracowanie własne

Sprawdzenie warunku zatrzymania zależy od konkretnego zastosowania tego algorytmu. Jeśli znana jest maksymalna wartość funkcji przystosowania zatrzymanie algorytmu może nastąpić po jej osiągnięciu. Jeśli maksymalna wartość jest nieznana, algorytm może zostać zatrzymany kiedy kolejne jego iteracje nie przynoszą zwiększenia przystosowania. Algorytm może zostać także zatrzymany po określonym czasie działania, lub po określonej ilości generacji. Po spełnieniu warunku zatrzymania następuje przejście do wyprowadzenia najlepszego chromosomu, jeśli warunek nie został spełniony kolejnym krokiem jest selekcja. W przykładzie znamy maksymalną wartość jaką może osiągnąć funkcja, w związku z czym warunek stopu może zostać ustawiony jako osiągnięcie przez którykolwiek chromosom wartości 500.³

Selekcja lub inaczej reprodukcja, polega na wyborze organizmów które będą brały udział w tworzeniu potomków do następnego pokolenia. Reprodukacja jest możliwa do zrealizowania algorytmicznie na wiele sposobów, najprostszym z nich jest symulacja „ruletki” – odpowiednio

³ Rutkowski L., op. cit., s. 240

skalibrowanej tarczy obrotowej. Każde jej pole odpowiada wartości wskaźnika przystosowania konkretnego ciągu w stosunku do sumy wszystkich wskaźników (procent całości w tab. 1). Proces reprodukcji polega na każdorazowym uruchomieniu ruletki. W przykładzie ciąg o numerze jeden ma wartość przystosowania równą 354, co odpowiada 48.9% całości. Przy każdym uruchomieniu ruletki prawdopodobieństwo wylosowania osobnika nr 1 wynosi zatem 0.489. Dzięki zastosowaniu mechanizmu ruletki lepiej przystosowane organizmy mają większą szansę na wprowadzenie swoich potomków do kolejnych pokoleń. Replika każdego ciągu wylosowanego do reprodukcji zostaje włączona do nowego pokolenia pośredniego, stanowiącego pulę rodzicielską kolejnego pokolenia.⁴



Rysunek 2 - Tarcza ruletki

Źródło: Opracowanie własne

Zastosowanie operatorów genetycznych do osobników wybranych metodą selekcji prowadzi do utworzenia nowej populacji, czyli populacji potomków. W podstawowym algorytmie genetycznym występują dwa podstawowe operatory genetyczne: krzyżowania oraz mutacji. W klasycznym algorytmie genetycznym operator krzyżowania pełni pierwszoplanową

⁴ Goldberg D. E., Algorytmy genetyczne i ich zastosowania, Wydawnictwo Naukowo-Techniczne, Warszawa 2003, s. 27-28

rolę, występuje bardzo często z prawdopodobieństwem $0.5 \leq p_k \leq 1$. Prawdopodobieństwo mutacji jest bardzo małe, rzędu $0 \leq p_m \leq 0.1$, co odpowiada sytuacji występującej w świecie rzeczywistym gdzie zachodzą one niezwykle rzadko. Mutacja może zachodzić zarówno na populacji rodziców przed krzyżowaniem, jak i na populacji potomków. Populacja rodziców to tymczasowa populacja składająca się z chromosomów wybranych w czasie selekcji i przeznaczonych do przetwarzania przez operatory genetyczne w celu stworzenia potomków.⁵

Krzyżowanie proste [ang. simple crossover] przebiega w dwóch fazach. W pierwszym etapie osobniki z puli rodzicielskiej są w sposób losowy kojarzone w pary. Następnie dla każdej ustalany jest punkt krzyżowania k – locus. Pozycję tą losuje się z jednakowym prawdopodobieństwem spośród $l-1$ początkowych pozycji (gdzie l to długość ciągu). Następnie w ciągach zamienia się miejscami wszystkie znaki od $k+1$ do l tworząc w ten sposób nowe ciągi które należą do generacji potomków.⁶ W przykładowym algorytmie przy założeniu że jako rodzice wylosowane zostały dwa pierwsze ciągi, a pozycja k w wyniku losowania przyjęła wartość 4 sytuacja wygląda następująco:

$$A_1 = 1100|10$$

$$A_2 = 0100|01$$

W wyniku krzyżowania następuje zamiana dwóch ostatnich pozycji w ciągach, co skutkuje powstaniem dwóch odmiennych ciągów wchodzących w skład nowego pokolenia:

$$A'_1 = 110001$$

$$A'_2 = 010010$$

Mutacja jest drugim z operatorów genetycznych który występuje w algorytmach genetycznych. Operator mutacji dokonuje zgodnie z prawdopodobieństwem mutacji p_m zmiany wartości genu chromosomu na przeciwną (z 0 na 1 lub z 1 na 0). Prawdopodobieństwo mutacji jest zazwyczaj bardzo małe. Mutacja polega na losowaniu liczby od 0 do 1 dla każdego z

⁵ Rutkowski L., op. cit., s. 242

⁶ Goldberg D. E., op. cit., s. 28

genów i wybraniu do mutacji genów których prawdopodobieństwo wynosi p_m lub mniej.⁷ Jako przykład rozważone zostanie prawdopodobieństwo otrzymanego wcześniej w wyniku krzyżowania genu $A'_1 = 110001$. Dla każdego z genów występujących w tym wylosowana jest liczba z zakresu od 0 do 1, a prawdopodobieństwo mutacji p_m wynosi 0.05.

geny przed mutacją	wylosowana wartość p_m	mutacja	geny po mutacji
1	0.76	nie	1
1	0.02	tak	0
0	0.98	nie	0
0	0.22	nie	0
0	0.73	nie	0
1	0.43	nie	1

Tabela 2 - Wartości genów w chromosomie przed i po mutacji

Źródło: Opracowanie własne

Zgodnie z ustalonym p_m mutacja zachodzi tylko dla genu na pozycji drugiej. W wyniku mutacji wartość chromosomu [110001] zostaje zmieniona na [100001].

⁷ Rutkowski L., op. cit., s. 243

Rozdział 2

Reprezentacja neuronów w matematyce i informatyce

2.1 Podstawowe informacje o budowie neuronu

Inspiracją do rozpoczęcia badań nad sztucznymi sieciami neuronowymi była chęć odtworzenia uproszczonego modelu ludzkiego układu nerwowego. Było to motywowane próbami stworzenia systemów zdolnych do samodzielnej nauki. Jednym z podstawowych elementów układu nerwowego ssaków jest komórka nerwowa nazywana neuronem. Budowa komórki nerwowej człowieka jest złożona i unikalna. Występują różnice budowy względem innych komórek w tym, że zawiera:

- wypustki,
- perykarion

Jest on zbudowany z ciała komórki zwanego perykarionem i odchodzących od niego 2 rodzajów wypustek: pewnej ilości dendrytów oraz 1 długiej wypustki zwanej, aksonem (neurytem).

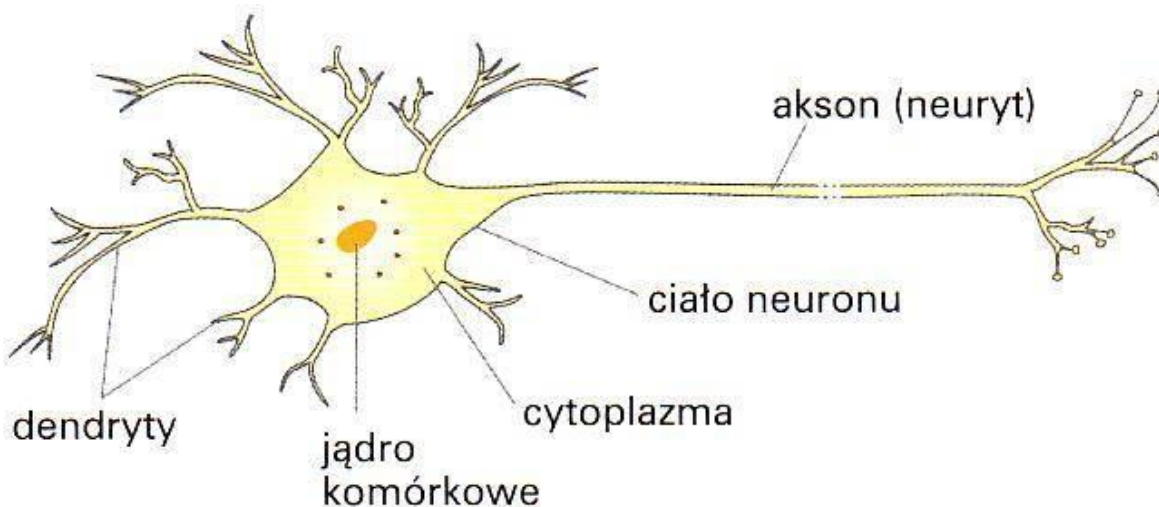
Wewnętrzne elementy które występują w komórce nerwowej człowieka są takie same, jak w spełniających te same funkcje komórkach zwierzęcych:

- cytoplazma – środowisko wewnętrzne komórki, w którym zachodzą reakcje chemiczne,
- jądro z chromosomami – centrum komórki zawierające większość jej materiału genetycznego,
- mitochondrium – odpowiedzialne za wytwarzanie energii wewnątrz komórki,
- aparat Golgiego – odpowiadający za modyfikacje białek.

Pod względem morfologicznym neurony dzieli się na neurony o jednym biegunie (występuje tylko akson), dwubiegunowe (1 akson i 1 dendryt) i posiadające wiele biegunów (1 akson i wiele dendrytów) Neuron składa się z:

- dendrytów, które zajmują się otrzymywaniem od receptorów sygnałów ze środowiska zewnętrznego komórki;
- jądra komórkowego, które przechowuje i powiela informację genetyczną;
- aksonu, który przekazuje impuls nerwowy do komórek receptorowych;
- synaps, które umożliwiają połączenie neuronu z innym neuronem, gruczołem albo mięśniem. Przekazują sygnał wyjściowy, np. uczucie pragnienia albo przyływ konkretnego hormonu.⁸

-



Rysunek 3 - schemat neuronu

Źródło: Encyklopedia Biologia, Wydanie: Kraków, 2, 2017

⁸ Longstaff A., *Neurobiologia*. Wydawnictwo Naukowe PWN, Warszawa 2002



Rysunek 4 - zdjęcie neuronu wykonane mikroskopem elektronowym

Źródło: Encyklopedia Biologia, Wydanie: Kraków, 2, 2017

Neuron charakteryzuje się wartością spoczynkową - w momencie gdy wartość potencjału jonów pomiędzy błoną wewnątrz a zewnątrzkomórkową wynosi w granicach ok. 70mV (w literaturze podawana jest to wartość w zakresie -65mV do -90mV ⁹) dochodzi do depolaryzacji błony, a w następstwie do propagacji przekazywania sygnału. Wysyłanie sygnałów pomiędzy neuronami odbywa się za pomocą tymczasowej zmiany w polaryzacji elektrycznej błony komórki nerwowej (neuronu). W wyniku konwersji sygnału z przekaźnictwa nerwowego na chemiczny, w komórce presynaptycznej uwalniane są neuroprzekaźniki (neurotransmitery). W zależności od stopnia pobudzenia komórki nerwowej wydzielana jest zmienna ilość neuroprzekaźników, zarówno pobudzających jak i hamujących, które dyfundując w przestrzeni synaptycznej ku komórce odbierającej sygnał nerwowy, łączą się ze swoistymi receptorami. Następnie impuls zmienia swój charakter z chemicznego na elektryczny i może być propagowany pomiędzy kolejnymi neuronami¹⁰. Wyładowanie neuronu uzależnione jest od sumy efektów oddziaływania synaps pokrywających obszary recepcyjne neuronu lub zależy od równowagi sygnałów pobudzeniowych i hamulcowych docierających do wzgórka aksonalnego. Stopniowalne pobudzeniowe (EPSPs) i hamulcowe (IPSPs) potencjały postsynaptyczne przewodzone są nagle i ubytkowo do wzgórka aksonalnego. Jeżeli suma odebranych

⁹ Krause M., *Człowiek i jego układ nerwowy*, Wydawnictwo Śląsk, Katowice 2003

¹⁰ Kossut M., *Mechanizmy plastyczności mózgu*. Wydawnictwo Naukowe PWN, Warszawa 1994

depolaryzacji i hiperpolaryzacji wystarczająca jest dla zdepolaryzowania błony komórkowej do poziomu zazwyczaj -55mV czyli progu pobudzenia neuronu na wzgórku aksonalnym wygenerowany zostaje potencjał czynnościowy. Potencjał czynnościowy jest chwilowym odwróceniem (ok. 1 milisekunda) potencjału błonowego neuronu z -70mV do około $+50\text{mV}$. Każdy neuron sumuje wszystkie stopniowalne potencjały postsynaptyczne (EPSPs i IPSPs) docierające do wzgórka aksonalnego i dokonuje wyładowania (lub nie) na podstawie wartości tych sum. Sumowanie tych pojedynczych sygnałów w jeden, większy sygnał nazywane jest integracją. Neurony integrują sygnały na dwa sposoby: w czasie i przestrzeni.

Neurony możemy podzielić po kątem pełnionych przez nie funkcji:

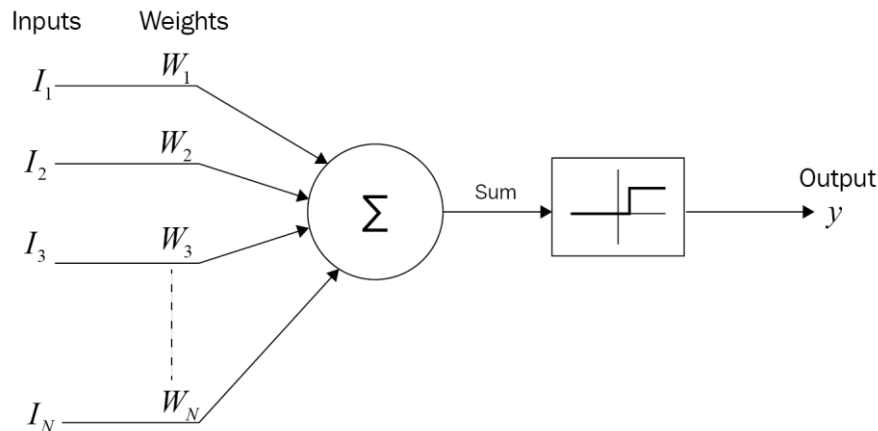
- neurony odpowiedzialne za czucie (sensoryczne) służą do odbierania bodźców ze środowiska zewnętrznego organizmu.
- neurony ruchowe, należące do grupy neuronów motorycznych - ich zadaniem jest doprowadzić sygnał do narządu wykonawczego, czyli efektora, takiego jak mięsień.
- neurony wisceroreceptorowe dostarczające organizmowi informacji odnośnie stanu narządów wewnętrznych.¹¹

W organizmie człowieka można znaleźć ok. 10^{12} neuronów. Komórki nerwowe przetwarzają impulsy z prędkością od 1 do 100 sygnałów na sekundę. Dzięki temu możemy obliczyć przybliżoną prędkość pracy mózgu. Wynosi ona 10^{18} operacji na sekundę, co wielokrotnie przewyższa możliwości obecnych superkomputerów.

2.2 Sztuczna reprezentacja neuronu za pomocą modeli matematycznych

Najprostszą reprezentacją neuronu w sztucznych sieciach neuronowych jest perceptron. Zostały one opracowane w latach 50. i 60. XX wieku. Prace Warrena McCullocha i Waltera Pittsa zainspirowały naukowca Franka Rosenblatta który przedstawił swój pomysł na reprezentację matematyczną komórek nerwowych.

¹¹ Górka T., Grabowska A., Zagrodzka J., *Mózg a zachowanie*. Wydawnictwo Naukowe PWN, Warszawa, 2012



Rysunek 5 - schemat projektu perceptronu

Źródło: Artificial Intelligence Example, Denis Rothman, 2018

Komórka McCullocha-Pittsa lub neuron McCullocha-Pittsa to model sztucznych neuronów opublikowany przez Warrena McCullocha i Waltera Pittsa w 1943 roku. Obaj chcieli zaprojektować uproszczony model rzeczywistych procesów w strukturach neuronowych, aby wyjaśnić, czy mózg naprawdę może obliczyć funkcje obliczalne według Turinga. Model neuronów McCullocha-Pittsa jest najprostszym modelem neuronowym w neuroinformatyce. Sztuczne sieci neuronowe zbudowane z komórek McCullocha-Pittsa mogą wykorzystywać tylko sygnały binarne. Każdy pojedynczy neuron może generować tylko 1 lub 0 jako wyjście. Sygnały hamujące mogą być przetwarzane w taki sam sposób, jak biologiczne sieci neuronowe. Każda komórka McCullocha-Pittsa ma dowolną liczbę rzeczywistą jako wartość progową.¹²

Gdy sygnały wejściowe mają wartość 1 lub 0, reguła aktywacji ma postać

$$y^{t+1} \equiv \begin{cases} 1 & \Leftrightarrow \sum w_i (x_i)^t \geq T \\ 0 & \Leftrightarrow \sum w_i (x_i)^t < T \end{cases}$$

gdzie $t = 0, 1, 2, \dots$ oznacza kolejne odcinki czasu, w_i zaś jest multiplikatywną wagą przypisaną połączeniu.

¹² McCulloch W., Pitts W., *A logical calculus of the ideas immanent in nervous activity*. In: *Bulletin of Mathematical Biophysics*, Bd. 5 (1943), S. 115–133

Sieci McCulloch-Pitts mogą być używane do symulacji bramek AND, OR i NOT. Tworzą więc pełną podstawę algebry Boole'a.

Jako komponent elektroniczny, neuron McCullocha-Pittsa posiada dużo większe możliwości niż prosta bramka AND lub OR. Dzięki niemu jako komponentowi można realizować bardzo wydajne obwody elektryczne, cechujące się mniejszym zużyciu komponentów i linii niż w przypadku konwencjonalnych bramek. Z tego powodu komórki McCullocha-Pittsa są nadal używane i badane w elektrotechnice, znane jako elementy wartości progowej. Model McCullocha-Pittsa cechuje prostota i prosta implementacja matematycznej definicji:

- Dopuszcza tylko binarne stany „0” i „1”
- Zakłada podział czasu na dyskretne odcinki i synchronizację neuronów w sieci
- Wagi i progi są niezmiennie

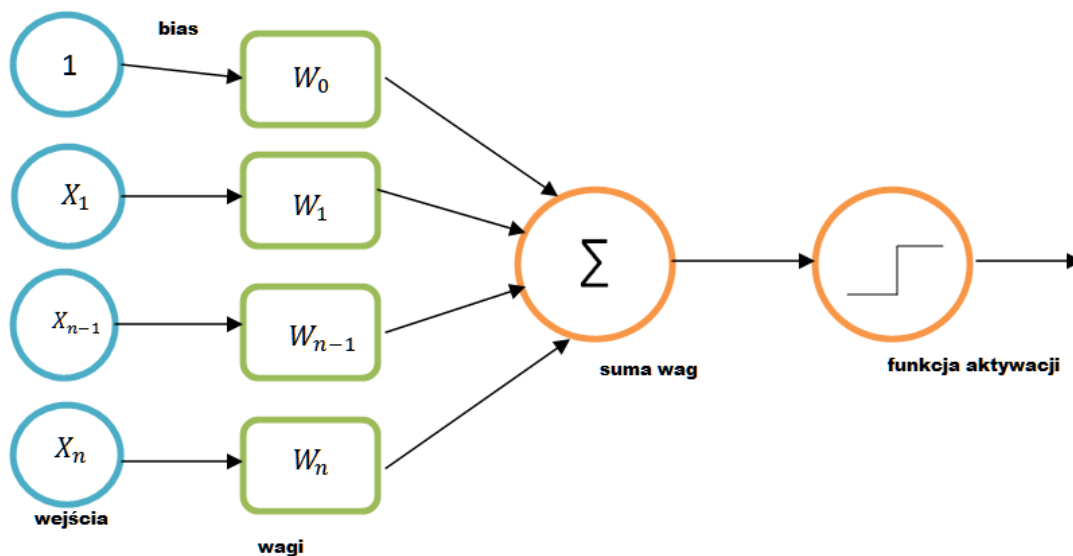
Przyjęte warunki nie pozwalają na zrealizowanie na nim sieci neuronowej zdolnej do klasyfikacji bardziej złożonych problemów. W 1949 roku Donald Hebb pomógł zrewolucjonizować sposób postrzegania sztucznych neuronów. W swojej książce *The Organization of Behaviour* zaproponował to, co stało się znane jako reguła Hebba. Twierdzi: „Kiedy akson komórki A jest wystarczająco blisko, aby wzbudzić komórkę B i wielokrotnie lub stale bierze udział w jej aktywowaniu, w jednej lub obu komórkach zachodzi pewien proces wzrostu lub zmiana metaboliczna, tak że wydajność A, jako jednej z komórki aktywującej B są zwiększone.”⁹ Jednym z postulatów Hebb’a było to, że kiedy dwa neurony aktywują się w tym samym momencie, następuje wzmocnienie połączenia pomiędzy nimi. Taka aktywność jest jedną z podstawowych operacji niezbędnych do uczenia się i zapamiętywania.

W przypadku sztucznego neuronu oznaczało to, że neuron McCullocha-Pittsa musiał zostać zmieniony, aby przynajmniej uwzględnić tę nową propozycję biologiczną. Zastosowana metoda polegała na modyfikacji wagi każdego z wejść. Zatem wejściu 1 można nadać większą lub mniejszą wagę w stosunku do całkowitej sumy progowej.

Frank Rosenblatt, używając neuronu McCullocha-Pittsa i odkrycia Hebba, przystąpił do opracowania pierwszego perceptronu. Ten perceptron, który mógł uczyć się poprzez ważenie danych wejściowych, odegrał kluczową rolę w późniejszym tworzeniu sieci neuronowych.

2.3 Perceptron

Perceptron jest konceptem matematycznym który umożliwia poprzez uczenie na zbiorze danych dobieranie odpowiednich wag w celu zwiększenia dokładności klasyfikacji danych. Zaproponowany przez Franka Rosenblatta model można zrozumieć tylko po uprzedniej analizie zmian w stosunku do wcześniej istniejącej reprezentacji neuronu. Jedyną różnicą między pierwotnym modelem neuronu i perceptronem to obecność wag w sieci. Rosenblatt badał również modele z kilkoma innymi różnicami, takimi jak ograniczając maksymalne dopuszczalne odchylenie jednostek. Niesatysfakcjonujące wyniki badań nad neuronem McCullocha-Pittsa doprowadziły do pomysłu udoskonalenia neuronu i rozwinięcia go o metody uczenia.



Rysunek 6 - schemat perceptronu
Źródło: Opracowanie własne

$$y^t(u) \equiv \begin{cases} 1 & \Leftrightarrow u^t \geq 0 \\ 0 & \Leftrightarrow u^t < 0 \end{cases}$$

Gdzie $u^i = \sum w_i x_i^t$, $w_i = 1$ lub $w_i = -1$, $i = 1, 2, 3 \dots n$.

Nauka perceptronu jest metodą klasyfikacji dla problemów z dwiema klasami (0 i 1), w których równanie liniowe może być użyte do rozdzielenia tych dwóch klas. Wagi algorytmu Perceptron należy oszacować na podstawie danych treningowych przy użyciu algorytmów propagacji wstecznej. Klasyczną metodą uczenia perceptronu jest metoda korekcji błędów. Jest to rodzaj uczenia z nadzorem, w którym waga wiązania nie zmienia się, dopóki aktualna odpowiedź perceptronu jest prawidłowa. W przypadku nieprawidłowej reakcji waga zmienia się o jeden, a znak (+/-) jest określany jako przeciwny do znaku błędu.

Perceptron wykonuje następujące kroki:

1. Pobiera dane wejściowe, mnoży je przez ich wagi i oblicza ich sumę. Wagi pozwalają perceptronowi ocenić względne znaczenie każdego z wyjść. Algorytmy sieci neuronowych uczą się, dobierając wagi, które skutkują dokładniejszą prognozą. Istnieje kilka algorytmów wykorzystywanych do precyzyjnego dostrojenia wag, z których najpopularniejszy nazywa się propagacją wsteczną.
2. Dodaje współczynnik odchylenia(bias), liczbę 1 pomnożoną przez wagę. Umożliwia to przesunięcie krzywej funkcji aktywacji w górę i w dół lub w lewo i w prawo na wykresie liczbowym. Umożliwia precyzyjne dostrojenie numerycznego wyjścia perceptronu.
3. Przekazuje sumę przez funkcję aktywacji. Funkcja aktywacji odwzorowuje wartości wejściowe na wymagane wartości wyjściowe. Na przykład, wartości wejściowe mogą wynosić od 1 do 100, a wartości wyjściowe mogą wynosić 0 lub 1. Funkcja aktywacji pomaga również perceptronowi uczyć się, kiedy jest częścią perceptronu wielowarstwowego. Pewne właściwości funkcji aktywacji, zwłaszcza jej nieliniowy charakter, umożliwiają trenowanie złożonych sieci neuronowych.

4. Wynikiem jest wyjście perceptronu w postaci pojedynczej liczby. Na wyjściu perceptronu znajduje się wynik funkcji klasyfikacji. W perceptronie wielowarstwowym wyjście perceptronów jednej warstwy jest wejściem kolejnej warstwy. Wyjście końcowych perceptronów, w ostatniej warstwie, jest ostateczną prognozą modelu uczenia się perceptronu.

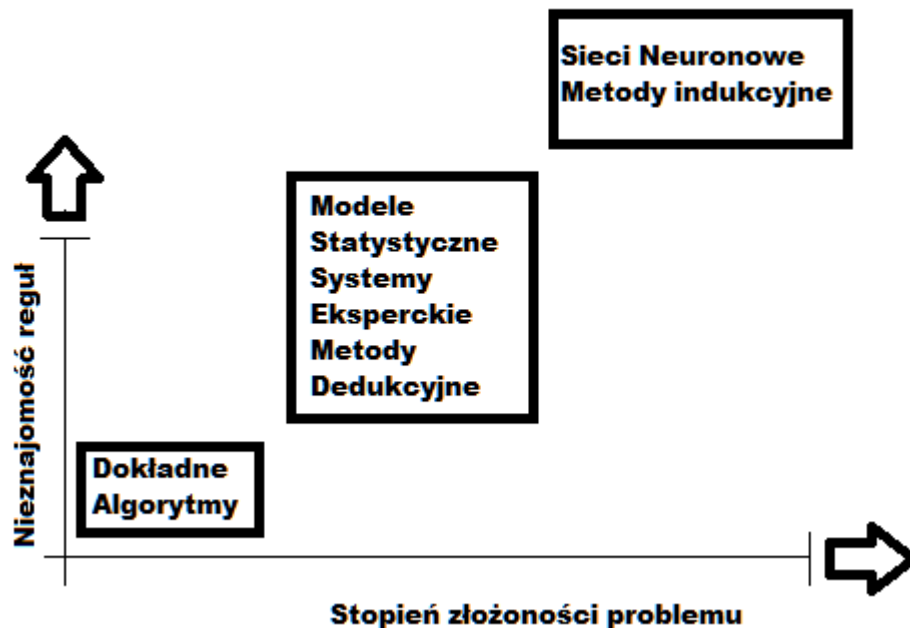
Wykonanie powyższej procedury dla wszystkich wektorów uczących nazywamy epoką uczenia. W przypadku uczenia perceptronu wykonujemy tyle epok, aż wszystkie przykłady uczące będą dobrze sklasyfikowane.

Rozdział 3

Sieć neuronowa

3.1 Złożoność obliczeniowa i ograniczenia algorytmów

Wraz z rozwojem informatyki i większą złożonością problemów stopień użyteczności perceptronu zaczął się zmniejszać. Perceptron jako funkcja może określić przynależność parametrów wejściowych do jednej z dwóch klas, poprzez wskazanie czy coś należy lub nie należy do pierwszej klasy. Wraz ze wzrostem złożoności problemu może pojawić się większa ilość końcowych zbiorów do których należy przyporządkować odpowiedź. Próby aplikacji perceptronów do problemów należących do klasy problemów Modeli Statystycznych lub Systemów Ekspertkich które posiadają kilka możliwych poprawnych odpowiedzi nie dawały wyników pozwalających na ich zastosowanie.



Rysunek 7 - Zależność złożoności problemu względem nieznajomości reguł

Źródło: Opracowanie własne

Perceptron jest reprezentacją klasyfikatora liniowego, co oznacza że nie będzie w stanie dokonać klasyfikacji zbiorów które nie są liniowo separowane. Jedyne możliwe wyniki to wszystkie odwzorowania, w których oddzielenie podobszaru z dziedziny X mającego formę otwartej podprzestrzeni dziedziny ograniczonej hiperpłaszczyzną. Zwiększenie ilości perceptronów lub gromadzenie ich w warstwy nie zmieni wyniku działania algorytmu który nie będzie w stanie wydzielić dwóch płaszczyzn. Proces uczenia pozwala ustalić graniczną hiperpłaszczyznę w dowolnym obszarze dziedziny, nie pozwala nam jednak na zmianę typu odwzorowania na inny niż liniowy, niezależnie od długości procesu uczenia.

Funkcja $y^t(u)$ jest funkcją liniową w całej swojej dziedzinie, co sprawia że obszar danych może zostać podzielony tylko linią prostą. Zastąpienie tej funkcji funkcją inną niż liniowa, bądź funkcją złożoną z różnych funkcji liniowych na pewnych odcinkach dziedziny pozwala nam na znacznie dokładniejsze i bardziej rozbudowane przyporządkowywanie elementów do zbiorów.¹³

3.2 rozwinięcie algorytmów do struktury sieci neuronowej

Różnicą pomiędzy wielowarstwową siecią perceptronów a siecią neuronową jest inna funkcja dokonująca aktywacji neuronu. Sygnał reprezentujący łączne pobudzenie neuronu przekształcany jest przez ustaloną funkcję aktywacji neuronu określaną jako funkcja przejścia neuronu. Wartość obliczona przez funkcję aktywacji jest ostatecznie wartością wyjściową która jako sygnał wyjściowy jest podawana na wyjście neuronu. Zachowanie neuronu jak i całej sieci neuronowej jest silnie uzależnione od rodzaju użytej funkcji aktywacji. Konieczność zastosowania nieliniowych funkcji aktywacji obejmuje konwersję liniowych sygnałów wejściowych i modeli na nieliniowe sygnały wyjściowe, co umożliwia zastosowanie wielomianów wyższego rzędu niż dla głębszych sieci. Istotną cechą nieliniowych funkcji aktywacji jest to, że są one różniczkowalne, w przeciwnym razie nie mogą być dopasowywane podczas wstecznej propagacji głębokich sieci neuronowych. W przypadku gdy funkcja aktywacji

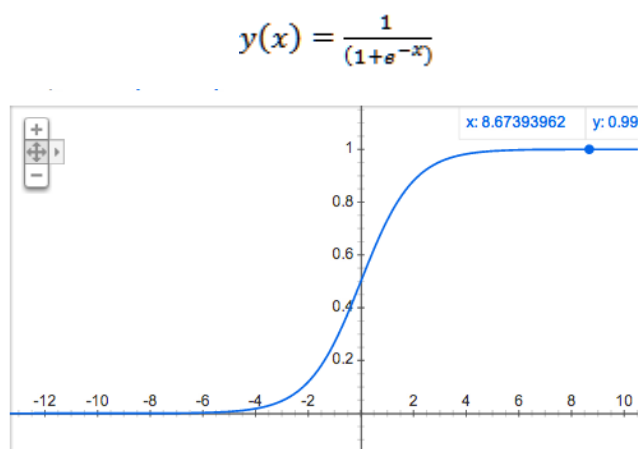
¹³ Cybenko, G., *Approximation by superpositions of a sigmoidal function Mathematics of Control, Signals, and Systems*, 1989, s.300–314.

jest funkcją nieliniową możliwe jest jeszcze dokładniejsze odwzorowanie działania uproszczonego modelu ludzkiego mózgu i jego zdolności poznawczych. Dzięki bardziej złożonym funkcjom aktywacji perceptrony wielowarstwowe mogą uczyć się klasyfikować bardziej złożone dane takie jak obrazy, nagrania mowy, nagrania dźwiękowe i video.

3.3 Rodzaje funkcji aktywacji

1. Funkcja sigmoidalna

Jest nieliniową funkcją aktywacji używaną głównie w sprzężonych sieciach neuronowych. Jest to różniczkowalna funkcja rzeczywista, zdefiniowana dla rzeczywistych wartości wejściowych i zawierająca wszędzie dodatnie pochodne o określonym stopniu gładkości. Funkcja sigmoidalna pojawia się w warstwie wyjściowej modeli głębokiego uczenia się i jest używana do przewidywania wyników opartych na prawdopodobieństwie. Funkcja sigmoidalna jest reprezentowana jako:



Rysunek 8 - Funkcja sigmoidalna

Źródło: opracowanie własne za pomocą narzędzi rysowania wykresów google

Pochodne funkcji sigmoidalnej są stosowane do uczenia algorytmów. Wykres funkcji sigmoidalnej ma kształt litery „S”.

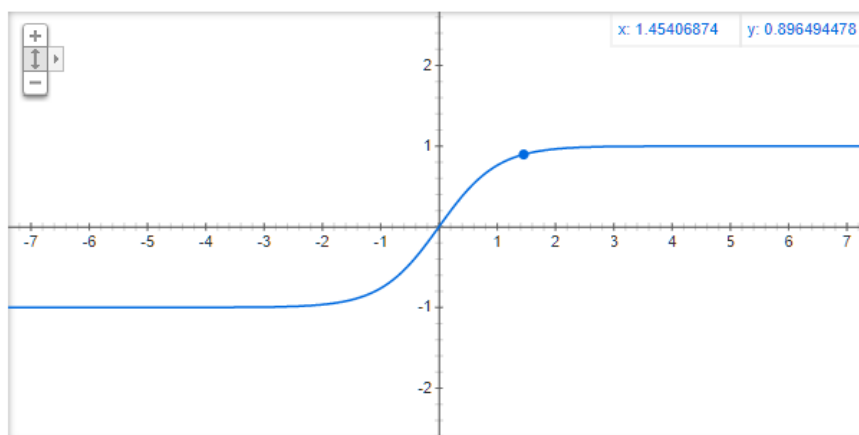
Niektóre z głównych wad funkcji sigmoidalnej obejmują nasycenie gradientu, powolną konwergencję, ostre nachylenie gradientu podczas propagacji wstecznej z głębszych warstw

ukrytych do warstw wejściowych oraz niezerowo wyśrodkowane wyjście, które powoduje propagację aktualizacji gradientu w różnych kierunkach.¹⁴

2. Hiperboliczna funkcja styczna, funkcja tanh

Jest gładsza, jest funkcją parzystą w dziedzinie od -1 do 1. W rezultacie wynik funkcji tanh jest reprezentowany przez:

$$\operatorname{tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



Rysunek 9 - tangens hiperboliczny

Źródło: opracowanie własne za pomocą narzędzi rysowania wykresów google

Funkcja tanh jest znacznie szerzej stosowana niż funkcja sigmoidalna, ponieważ zapewnia lepszą wydajność uczenia wielowarstwowych sieci neuronowych. Największą zaletą funkcji tanh jest to, że dla argumentu 0 przyjmuje wartość 0, wspierając w ten sposób proces wstecznej propagacji. Funkcja tanh była najczęściej używana w powtarzających się sieciach neuronowych do przetwarzania języka naturalnego i zadań rozpoznawania mowy.¹⁵

Jednak funkcja tanh również ma ograniczenia - podobnie jak funkcja sigmoidalna nie może rozwiązać problemu znikającego gradientu. Ponadto funkcja tanh może osiągnąć gradient 1 tylko

¹⁴ Goodfellow I., Bengio Y., Courville A., *Deep learning*. MIT Press, 2016.

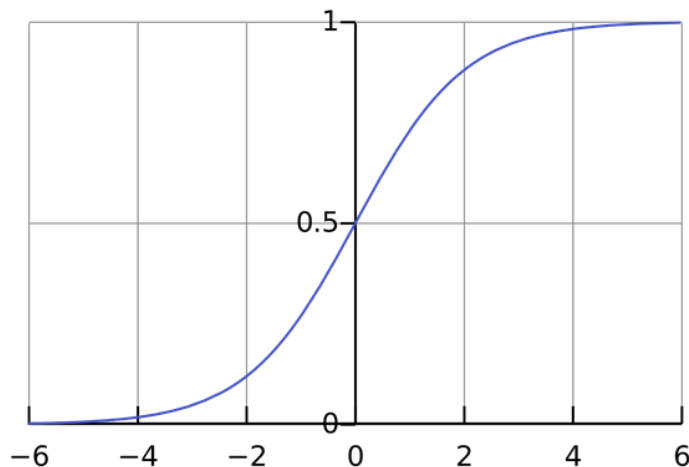
¹⁵ LeCun Y., Bengio Y., Hinton G., *Deep learning*, Nature, 2015. s. 436–444

wtedy, gdy wartość wejściowa wynosi 0 (x wynosi zero). W rezultacie funkcja może wytwarzać nieaktywne neurony podczas procesu obliczania.

3. Funkcja Softmax

Funkcja softmax to kolejny typ funkcji używanej w sieciach neuronowych do obliczania rozkładu prawdopodobieństwa z wektora liczb rzeczywistych. Ta funkcja generuje dane wyjściowe w zakresie od 0 do 1, przy czym suma prawdopodobieństw jest równa 1. Funkcja softmax jest przedstawiona w następujący sposób:

$$\sigma(x)_i = \frac{e^{\beta x_i}}{\sum_{j=1}^n e^{\beta x_j}}$$



Rysunek 10 - funkcja softmax

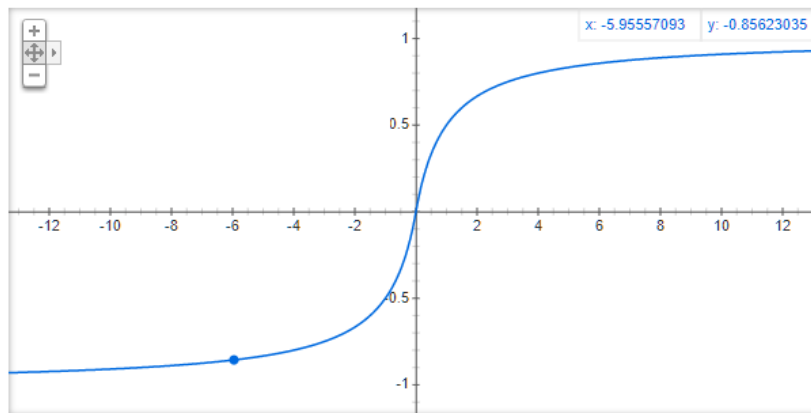
Źródło: opracowanie własne za pomocą narzędzi rysowania wykresów google

Ta funkcja jest używana głównie w modelach wieloklasowych, w których zwraca prawdopodobieństwa każdej klasy, przy czym klasa docelowa ma najwyższe prawdopodobieństwo. Podstawowa różnica między sigmoidalną funkcją a funkcją softmax polega na tym, że podczas gdy pierwsza jest używana w klasyfikacji binarnej, druga jest używana do klasyfikacji wieloczynnikowej.

4. Funkcja Softsign

Funkcja softsign to kolejna funkcja używany w obliczeniach sieci neuronowych. Choć dotyczy to głównie problemów obliczeń regresji, obecnie jest również używany w aplikacjach do zamiany tekstu na mowę opartych o algorytmy głębokiego uczenia. Jest to wielomian kwadratowy, reprezentowany przez:

$$f(x) = \left(\frac{x}{|x| + 1} \right)$$



Rysunek 11 - funkcja softsign

Źródło: opracowanie własne za pomocą narzędzi rysowania wykresów google

Główna różnica między funkcją softsign a funkcją tanh polega na tym, że w przeciwieństwie do funkcji tg, która zbiega wykładniczo, funkcja softsign zbiega do nieskończoności w postaci wielomianu.¹⁶

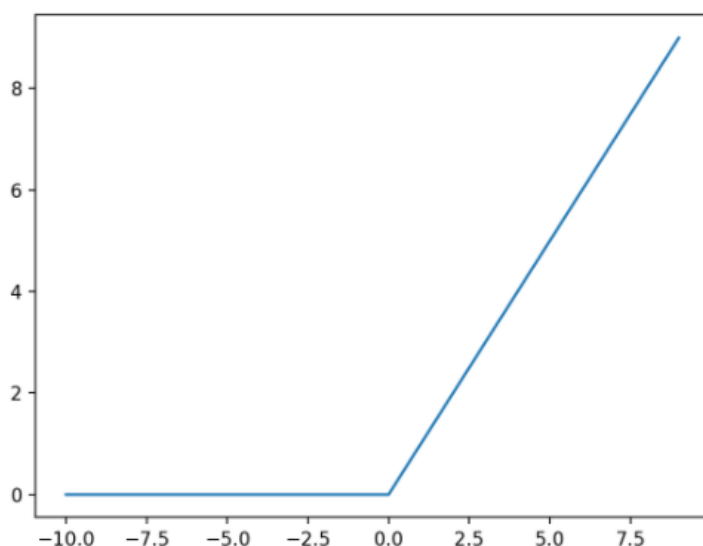
5. Funkcja rektyfikowanej jednostki liniowej (ReLU).

¹⁶ Turian J., Bergstra J., Bengio Y., *Quadratic features and deep architectures for chunking*, in Proceedings of Human Language Technologies: The 2009 Annual Conference of the North American Chapter of the Association for Computational Linguistics, vol. Companion Volume, 2009, s. 245–248.

Funkcja aktywacji rektyfikowanej jednostki liniowej (ReLU) została zaproponowana przez Vinoda Naira i Geoffrea E. Hinton z University of Toronto w 2010 i od tamtej pory jest najbardziej szeroko stosowaną funkcją aktywacji dla aplikacji do głębokiego uczenia z najlepszymi możliwymi wynikami.¹⁷ W porównaniu z innymi AF, takimi jak funkcje sigmoid i tanh, funkcja ReLU zapewnia znacznie lepszą wydajność i generalizację w głębokim uczeniu się. Jest to funkcja prawie liniowa, która zachowuje właściwości modeli liniowych, co ułatwia ich optymalizację za pomocą gradientu.

Funkcja ReLU wykonuje operację progową na każdym elemencie wejściowym, w której wszystkie wartości mniejsze od zera są ustawiane na zero. Tak więc ReLU jest reprezentowany jako:

$$f(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$



Rysunek 12 - Funkcja ReLU

Źródło: opracowanie własne za pomocą narzędzi rysowania wykresów google

¹⁷ Nair V., Hinton G.E., *Rectified linear units improve restricted boltzmann machines*, Haifa, 2010, str. 807–814

Poprzez prostowanie wartości wejść mniejszych od zera i ustawienie ich na zero, funkcja ta eliminuje problem zanikania gradientu obserwowany we wcześniejszych typach funkcji aktywacji (sigmoid i tg).

Najistotniejszą zaletą korzystania z funkcji ReLU w obliczeniach jest to, że gwarantuje ona szybsze obliczenia - nie oblicza wykładników i podziałów, zwiększając w ten sposób ogólną prędkość obliczeń.

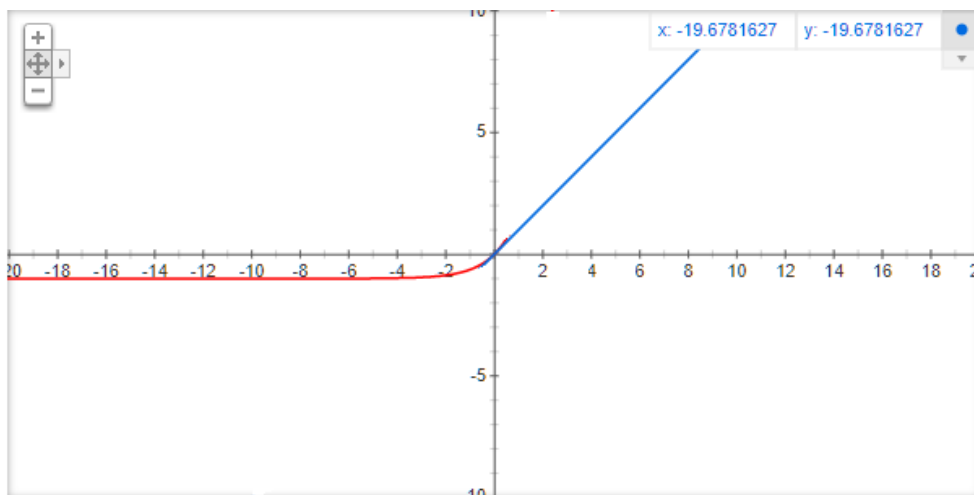
6. Funkcja wykładniczych jednostek liniowych (ELU)

Funkcja wykładniczych jednostek liniowych (ELU) to AF, który jest również używany do przyspieszenia uczenia sieci neuronowych (podobnie jak funkcja ReLU). Największą zaletą funkcji ELU jest to, że może ona wyeliminować problem znikającego gradientu poprzez wykorzystanie tożsamości dla wartości dodatnich i poprzez poprawę charakterystyki uczenia się modelu.

ELU mają wartości ujemne, które powodują, że średnia aktywacja jednostki zbliża się do zera, zmniejszając w ten sposób złożoność obliczeniową i poprawiając szybkość uczenia się. ELU jest doskonałą alternatywą dla ReLU - zmniejsza przesunięcia odchylenia, przesuwając średnią aktywację do zera podczas procesu treningu.

Wykładnicza funkcja jednostki liniowej jest reprezentowana jako:

$$f(x) = \begin{cases} x & \Rightarrow x > 0 \\ \alpha \cdot e^x - 1 & \Rightarrow x \leq 0 \end{cases}$$



Rysunek 13 - funkcja ELU

Źródło: opracowanie własne za pomocą narzędzi rysowania wykresów google

Tutaj „ α ” jest parametrem ELU, który kontroluje punkt nasycenia dla ujemnych danych wejściowych, który zwykle jest ustawiony na 1,0. Jednak funkcja ELU ma pewne ograniczenie - nie jest wyśrodkowana na zero.

Istnieją też funkcje stosowane zdecydowanie rzadziej, takie jak:

- Sinus - funkcja aktywacji z dziedziny funkcji trygonometrycznych może być zastosowana przy klasyfikowaniu danych rozłożonych radialnie, a także może aproksymować spektralne rozwinięcia sygnału wejściowego.
- Liniowa z nasyceniem - jest to wersja funkcji sigmoidalnej która jest połączona liniowymi odcinkami. Opisana funkcja aktywacji umożliwia bardzo szybkie działanie nauczonej sieci, ale służy głównie do budowy sieci cechujących się niską sprawnością uczenia.
- Progowa - wartości na wyjściu funkcji wynoszą wyłącznie 1 lub 0. w zależności od tego czy wartość funkcji potencjału postsynaptycznego jest dodatnia, czy też ujemna. Jednym z zastosowań tej funkcji jest modelowanie prostych perceptronów.¹⁸

¹⁸ Tadeusiewicz R., Szaleniec M., *Leksykon sieci neuronowych*, Wydawnictwo Fundacji „Projekt Nauka”, Wrocław 2015, s. 68

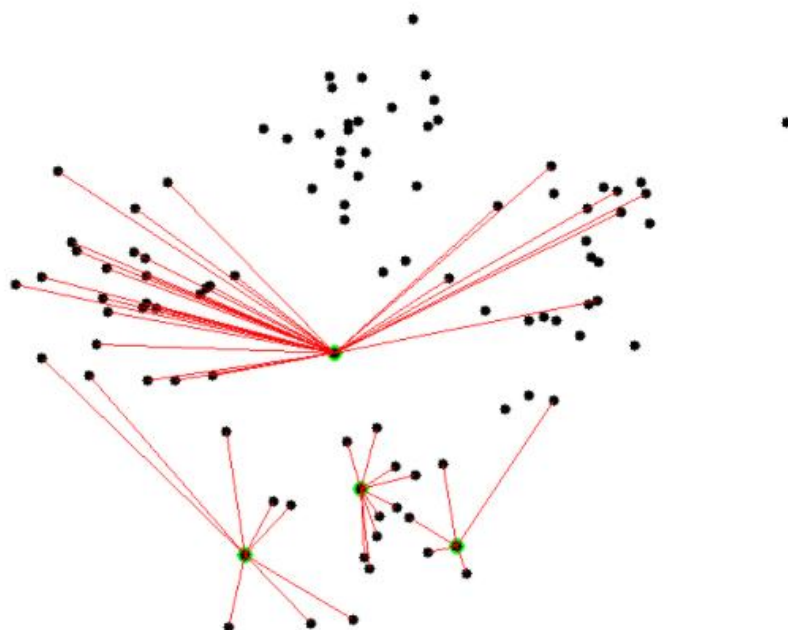
3.4 Rodzaje i struktury sieci neuronowych

1. Sieć neuronowa z przekazywaniem do przodu - Sztuczny neuron:

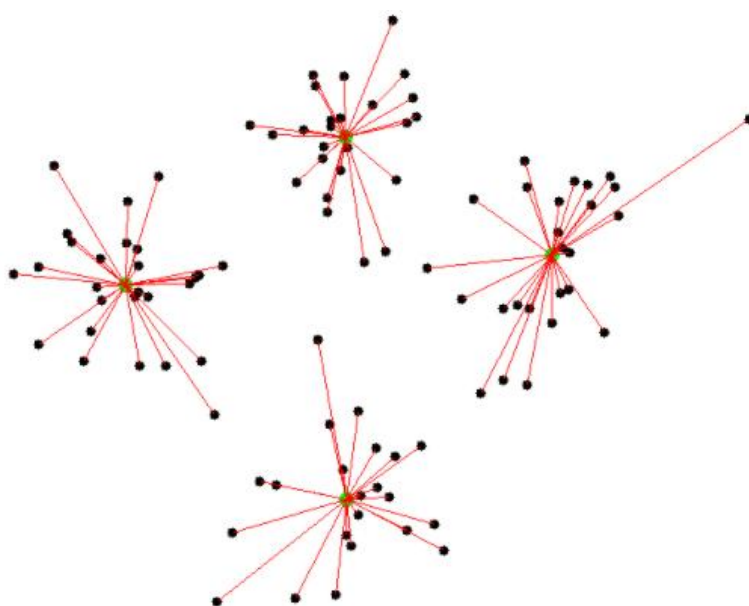
Ta sieć neuronowa jest jedną z najprostszych reprezentacji sztucznych neuronów, gdzie dane lub dane wejściowe są przekazywane w jednym kierunku. Dane przechodzą przez węzły wejściowe gdzie są sumowane oraz wyjście na węzłach wyjściowych. Ta sieć neuronowa może mieć strukturę złożoną z ukrytych warstw o różnej liczbie neuronów. Model jest pobudzany poprzez podanie danych na wejście i nie posiada algorytmu propagacji wstecznej przy użyciu klasyfikującej funkcji aktywacji. W sztucznym neuronie suma iloczynów wejść i wag jest obliczana i podawana na wyjście. Wyjście jest brane pod uwagę, jeśli jest powyżej pewnej wartości progu i neuron uruchamia się z aktywowanym wyjściem. W przypadku gdy neuron nie uruchamia się, emitowana jest wartość dezaktywowana. Zastosowanie sieci neuronowych typu Sztuczny Neuron można znaleźć w przekształcaniu sygnałów video i rozpoznawaniu mowy, gdzie kategoryzowanie klas docelowych jest złożonym procesem. Tego typu sieci neuronowe posiadają dużą czułość na dane posiadające wysoki stosunek S/N, szumu do wartości użytecznej.

2. Radialna funkcja bazowa:

Sieci o architekturze radialnej funkcji bazowej rozważają odległość punktu w odniesieniu do środka zbioru danych. Funkcje te mają dwie warstwy, najpierw gdzie dane są łączone w warstwie wewnętrznej, a następnie wyjścia z tych neuronów są brane pod uwagę podczas obliczania tego samego wyjścia w następnej warstwie. Skuteczność modelu zależy od maksymalnego zasięgu lub promienia okręgu w klasyfikowaniu punktów do różnych kategorii. Jeśli punkt znajduje się w promieniu lub w jego pobliżu, prawdopodobieństwo, że nowy punkt przyporządkowany danym zostanie zaklasyfikowany do danej klasy jest wysokie.



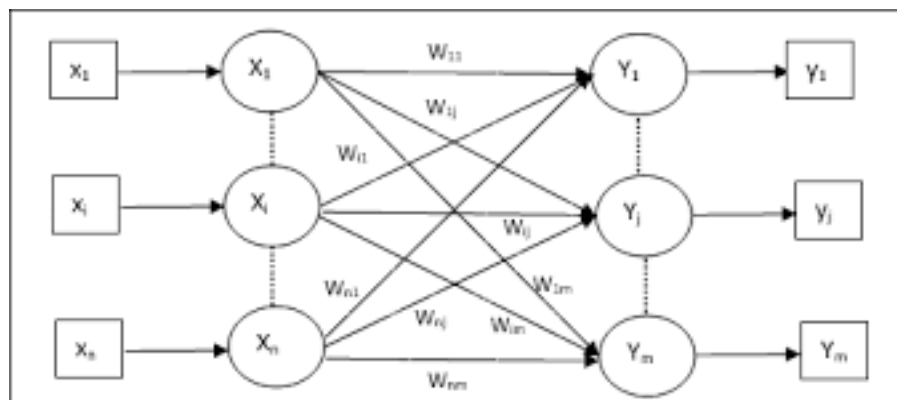
Rysunek 14 - Radialna funkcja bazowa na początku analizy danych
Źródło: opracowanie własne



Rysunek 15 - Radialna funkcja bazowa po dokonaniu organizacji danych
Źródło: opracowanie własne

3. Samoorganizująca się sieć neuronowa Kohonena:

Celem sieci Kohonena jest wprowadzenie na obszar danych wektorów o dowolnym wymiarze do sieci neuronowej traktowanej dalej jako dyskretnej mapy składającej się z neuronów. Mapa jest siecią która została wcześniej nauczona z nadzorem w celu utworzenia organizacji danych szkoleniowych. Obejmuje ona jeden lub dwa wymiary. Podczas treningu mapy położenie neuronu pozostaje stałe, ale wagi różnią się w zależności od wartości. Ten proces samoorganizacji składa się z różnych części, w pierwszej fazie każda wartość neuronu jest inicjalizowana z małą wagą i wektorem wejściowym.



Rysunek 16 - Schemat blokowy Samoorganizującej się sieci neuronowej Kohonena

Źródło: opracowanie własne

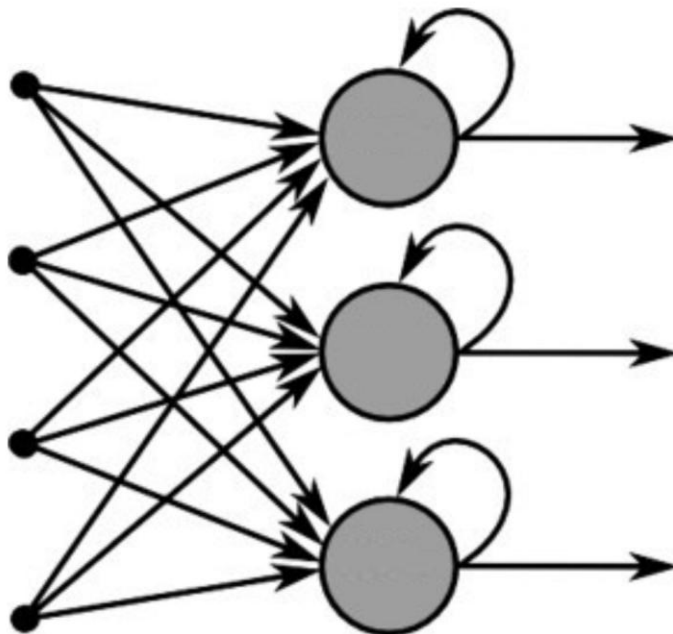
W drugiej fazie, neuron znajdujący się najbliżej punktu jest neuronem o najwyższej skuteczności a neurony połączone z tym neuronem również będą poruszać się w kierunku punktu. Odległość pomiędzy punktem a neuronami jest obliczana na podstawie odległości na płaszczyźnie euklidesowej, neuron z najmniejszą odległością jest taktowany jako obiekt o najwyższej skuteczności. Poprzez kolejne iteracje, wszystkie punkty są grupowane, a każdy neuron reprezentuje każdy rodzaj klastra.

Sieć neuronowa Kohonena jest używana do rozpoznawania wzorców w danych. Jej zastosowanie można znaleźć w analizie medycznej do grupowania danych w kategorie.¹⁹

¹⁹ Grabowski M., *Zastosowanie samoorganizujących się map cech Kohonena w analizie danych*, 1997

4. Sieć neuronowa rekurencyjna (RNN):

Rekurencyjna sieć neuronowa działa na zasadzie zapisywania wyjścia jednej warstwy i zapętlania wyjścia z powrotem na wejście tej samej warstwy, aby zwiększyć szanse na przewidzenie wyniku danej warstwy. Pierwsza warstwa jest tworzona podobnie jak w sieci neuronowej z przekazywaniem do przodu z obliczenia iloczynu sumy wag i cech. Proces rekurencyjnej sieci neuronowej rozpoczyna się po dokonaniu tych obliczeń z przekazania sygnału z jednej warstwy na następne neurony które wraz ze sprzężeniem zwrotnym otrzymają informacje, które posiadały w poprzedniej iteracji. Dzięki zastosowaniu takiego rozwiązania każdy neuron działa jak komórka rzeczywistej pamięci krótkotrwałej podczas wykonywania obliczeń. W trakcie trwania tego procesu następuje aktualizacja wagi danego połączenia. W przypadku gdy przewidywanie wartości wagi jest błędne, funkcja odpowiadająca za uczenie się sieci wprowadza zmiany wartości wag wraz z każdą iteracją. Metodą wspomagającą przewidywanie jest algorytm wstecznej propagacji. Zastosowanie rekurencyjnych sieci neuronowych można znaleźć w modelach konwersji tekstu na mowę. Inną nazwą tej sieci jest też „Long short-term memory (LSTM)”



Rysunek 17 - Schemat rekurencyjnej sieci neuronowej

Źródło: opracowanie własne

5. Neuronowa sieć konwolucyjna:

Konwolucyjna sieć neuronowa, to wyspecjalizowany typ modelu sieci neuronowej przeznaczony do pracy z dwuwymiarowymi danymi obrazu. Centralnym elementem tej sieci neuronowej jest warstwa przeprowadzająca operację splotu (konwolucji), która nadaje sieci jej nazwę. W tym kontekście spłot jest operacją liniową, która polega na pomnożeniu zestawu wag wraz z wejściem, podobnie jak w tradycyjnej sieci neuronowej. Biorąc pod uwagę, że technika ta została zaprojektowana do dwuwymiarowego wprowadzania danych, mnożenie jest wykonywane między tablicą danych wejściowych a dwuwymiarową tablicą wag, zwaną filtrem lub jądrem. Filtr jest macierzą złożoną z elementów o wartości mniejszej niż dane wejściowe, a rodzaj mnożenia zastosowanego między fragmentem o rozmiarze filtra wejścia a filtrem jest iloczynem skalarnym.

$$\vec{u} \circ \vec{v} = |\vec{u}| |\vec{v}| \cos \angle(\vec{u}, \vec{v})$$

Rysunek 18 - Iloczyn skalarny wektorów

Źródło: opracowanie własne

Użycie filtra mniejszego niż wartość wejściowa jest celowe, ponieważ umożliwia wielokrotne pomnożenie tego samego filtra (zestawu wag) przez tablicę wejściową w różnych punktach na wejściu. W szczególności filtr jest stosowany systematycznie do każdej nakładającej się części lub obszaru danych wejściowych o rozmiarze filtra. Jeśli filtr jest przeznaczony do wykrywania określonego typu cechy na wejściu, wówczas zastosowanie tego filtra systematycznie na całym obrazie wejściowym daje filtrowi możliwość odkrycia tej cechy w dowolnym miejscu obrazu.

6. Modułowa sieć neuronowa:

Modułowe sieci neuronowe mają zbiór różnych sieci pracujących niezależnie i przyczyniając się do wyjścia. Każda sieć neuronowa ma zestaw wejść, które są unikalne w porównaniu do innych sieci budujących i wykonujących zadania cząstkowe. Sieci te nie wchodzi w interakcje lub sygnalizują sobie nawzajem w realizacji zadań.

Zaletą modułowej sieci neuronowej jest to, że rozбивa ona duży proces obliczeniowy na mniejsze elementy zmniejszające jego złożoność. Podział ten pomoże w zmniejszeniu liczby połączeń i redukuje interakcję tych sieci ze sobą, co z kolei zwiększy szybkość obliczeń. Jednakże czas przetwarzania będzie zależał od liczby neuronów i ich zaangażowania w obliczanie wyników.²⁰

²⁰ Osowski S., *Sieci neuronowe do przetwarzania informacji*. Wydawnictwo OW PW, Warszawa 2013, s. 422.

Rozdział 4

Metody optymalizacji i treningu sieci neuronowych

4.1. Optymalizacja

Optymalizacja matematyczna zajmuje się problemem znajdowania liczbowych ekstremów (minimów, maksimów lub zer) funkcji. Skala problemu optymalizacyjnego jest w dużej mierze określona przez wymiarowość problemu, tj. liczbę zmiennych skalarnych, na których wykonywane jest wyszukiwanie. Uczenie maszynowe bazuje na niektórych metodach optymalizacji. Wiele problemów uczenia się jest sprowadzonych do minimalizacji pewnej funkcji straty przedstawionej algorytmowi za danych treningowych. Funkcje straty wyrażają rozbieżność pomiędzy przewidywaniami trenowanego modelu a rzeczywistymi instancjami problemu. Zadania optymalizacji dzielimy na dwie podstawowe klasy: optymalizację statyczną (sprowadzającą się do poszukiwania ekstremum funkcji) oraz optymalizację dynamiczną, sprowadzającą się do poszukiwania ekstremum funkcjonału. Metody optymalizacyjne możemy też podzielić na typy algorytmów do szukania optimum. Możemy wyróżnić metody deterministyczne i losowe znajdowania optimum, metody znajdowania optimum z ograniczeniami i bez ograniczeń, metody optymalizacji prostej i wielopoziomowej.

4.2. Metody uczenia i trenowania sieci neuronowych

Większość poznanych obecnie statystycznych problemów uczenia się należy do jednej z dwóch kategorii: z nadzorem lub bez nadzoru. Wiele klasycznych statystycznych metod uczenia się, takich jak regresja liniowa i regresja logistyczna, a także bardziej nowoczesne metody optymalizacji skuteczności działają w nadzorowanej domenie uczenia się. W sztucznych sieciach neuronowych, uczenie odnosi się do metody modyfikowania wag połączeń między węzłami określonej sieci. Uczenie jest procesem, w którym losowo dobrane parametry takie jak wagi i

liczba neuronów w każdej warstwie sieci neuronowej są dostosowywane poprzez ciągły proces symulacji przez środowisko, w którym sieć jest osadzona. Szybkość uczenia jest definiowana jako szybkość, z jaką sieć ulega adaptacji. Typ uczenia jest określany przez sposób, w jaki następuje zmiana parametrów. Uczenie może być skategoryzowane jako

1. Uczenie z nadzorem (supervised learning)

Uczenie nadzorowane lub uczenie asocjacyjne, w którym sieć jest trenowana poprzez dostarczanie jej wcześniej przygotowanych danych. Sieć wyprowadza funkcję z oznaczonych danych treningowych składających się z zestawu przykładów szkoleniowych. Następuje „uczenie” się funkcji, która przyporządkowuje dane wejściowe na dane wyjściowe na podstawie przykładowych, celowo dobranych par wejście-wyjście. Optymalna implementacja pozwoli algorytmowi poprawnie określić klasy dla nieskateryzowanych danych. Wymaga to od algorytmu uczącego się odczytania ogólnego wzorca z danych szkoleniowych w sposób pozbawiony błędu indukcyjnego (ang. bias). Informacje trafiające na neurony wejściowe są całkowicie sklasyfikowane i na tej podstawie sieć jest w stanie dobrać wagi tak by konkretne rodzaje wzorców pobudzały wykrywające je fragmenty sieci.²¹ Dane mogą być dostarczone przez zewnętrznego nauczyciela, lub przez system który nadzoruje się sam i zawiera sieć neuronową. W uczeniu nadzorowanym, wzorzec jest dostępny w celu wskazania, czy system czy system działa poprawnie, wskazania poprawnej odpowiedzi, walidacji akceptowalnej odpowiedzi systemu, lub wskazania wielkość błędu klasyfikacji poprzez sieć²². Jest to przeciwieństwo do uczenia nienadzorowanego, gdzie sieć nie posiada wzorca i uczenie się musi polegać na wskazówkach uzyskanych heurystycznie przez system badający zewnętrzne dane lub parametry środowiska. Sieć neuronowa musi być skonfigurowana w taki sposób, aby zastosowanie zestawu danych wejściowych dawało określony zestaw danych wyjściowych. Możemy wymienić różne metody ustawiania siły połączeń pomiędzy poszczególnymi neuronami. Jedną z nich sposobem jest jawne ustawienie wag na podstawie wyników innych sieci.

²¹ Rostamizadeh A., Talwalkar A. (2012) *Foundations of Machine Learning*, The MIT Press, 2012

²² Mirjalili V., Sawka K., *Python : uczenie maszynowe*, Wydawnictwo Helion, Gliwice 2019

Najpopularniejsze algorytmy uczenia z nadzorem to:

- Wsparcie maszyn wektorowych
- regresja liniowa
- regresja logistyczna
- naiwny Bayes
- liniowa analiza dyskryminacyjna
- drzewa decyzyjne
- Algorytm k-najbliższego sąsiada
- Sieci neuronowe (perceptron wielowarstwowy)
- Uczenie się podobieństwa

2. Uczenie bez nadzoru (unsupervised learning)

Uczenie nienadzorowane to metoda w której sieć jest szkolona do reagowania na skupiska wzorców w danych wejściowych. Metoda zakłada że system ma na celu odkrycie statystycznie istotnych cech populacji danych wejściowych. W przeciwieństwie do typu uczenia nadzorowanego, nie ma tu z góry założonego zbioru kategorii, do których mają być zaklasyfikowane wzorce; sieć musi opracować własną reakcję i konfigurację wag na nieznaną dane wejściowe.

Niektóre z najczęściej używanych algorytmów w uczeniu się bez nadzoru obejmują:

- grupowanie,
- wykrywanie anomalii,
- sieci neuronowe (Neocognitron)²³
- podejścia do uczenia modeli utajonych zmiennych.²⁴

²³ LeCun Y., Bengio Y., Hinton G., op.cit., s. 521

²⁴ Hastie T., Tibshirani R., Friedman J., *The Elements of Statistical Learning: Data mining, Inference, and Prediction*, Wydawnictwo Springer, New York 2009, s. 485–586.

Metody grupowania obejmują hierarchiczne grupowanie, metodę k-średnich, natomiast metody wykrywania anomalii obejmują lokalny współczynnik zmiennych. Podejścia do uczenia się modeli zmiennych ukrytych, takich jak algorytm oczekiwania i maksymalizacji, metoda separacji sygnałów (analiza składowych głównych, analiza składowych niezależnych, nieujemna faktoryzacja macierzy, dekompozycja wartości osobliwych)²⁵

3. Uczenie wzmacnione (reinforcement learning)

Uczenie wzmacniające jest rodzajem uczenia się, które może być uważane za pośrednią formę dwóch powyższych typów uczenia. W tym przypadku maszyna ucząca się wykonuje pewne działania na środowisko z którego otrzymuje dane i otrzymuje odpowiedź zwrotną z tego środowiska. System uczący się ocenia swoje działanie jako dobre (nagradzające) lub złe (karzące) na podstawie reakcji środowiska i wartości funkcji błędu. Na podstawie tych wartości odpowiednio dostosowuje swoje parametry w celu osiągnięcia wyższej skuteczności. Regulacja parametrów jest kontynuowana do momentu osiągnięcia stanu równowagi, po którym nie będzie już żadnych zmian. po którym nie będzie więcej zmian w jego parametrach. Samoorganizujące się uczenie neuronowe może być zakwalifikowane do tego typu uczenia. W uczeniu nadzorowanym, wzorzec jest dostępny, aby wskazać, czy system działa poprawnie, aby wskazać pożądaną odpowiedź, aby walidować akceptowalność odpowiedzi systemu, lub wskazywać wielkość błędu w działaniu systemu. Jest to w przeciwieństwie do uczenia nienadzorowanego, gdzie sieć nie posiada wzorca i uczenie się musi polegać na wskazówkach uzyskanych heurystycznie przez system badający zewnętrzne dane lub parametry środowiska. Sieć neuronowa musi być skonfigurowana w taki sposób, aby zastosowanie zestawu danych wejściowych dawało pożądaną zestaw danych wyjściowych. Istnieją różne metody ustawiania siły połączeń. Jedną z nich sposobem jest jawne ustawienie wag, przy użyciu a priori wiedzy. Innym sposobem jest "trenowanie" sieci neuronowej poprzez podając jej wzorce i pozwalając na zmianę jej wag zgodnie z pewną regułą²⁶

²⁵ Hinto G.; Sejnowski, T., *Unsupervised Learning: Foundations of Neural Computation*, MIT Press, 1999

²⁶ Griewank, A. *Who Invented the Reverse Mode of Differentiation?*, 2012, s. 389–400

4.3. Metoda gradientu prostego

Modele połączeń neuronowych sieci zostały z powodzeniem zastosowane w szerokim zakresie problemów. Chociaż dostępnych jest wiele różnych odmian algorytmów uczenia się, większość z nich - w tym popularny algorytm uczenia się z propagacją wsteczną - należą do typu zejścia gradientowego. Dla danej architektury sieci zwykle zaczyna się od funkcji błędu, która jest sparametryzowana przez wagi (siły połączenia pomiędzy jednostek) w sieci. Gradient funkcji błędu w odniesieniu do każdej wagi jest następnie obliczana i wagi są modyfikowane wzdłuż kierunku w dół gradientu w celu zmniejszenia błędu. Zejście gradientowe jest jednym z najpopularniejszych algorytmów do przeprowadzania optymalizacji i zdecydowanie najpowszechniejszym sposobem optymalizacji sieci neuronowych.²⁷ Algorytmy te są jednak często używane jako optymalizatory czarnej skrzynki, ponieważ trudno jest znaleźć praktyczne wyjaśnienie ich mocnych i słabych stron. Istnieją trzy warianty spadku gradientu, które różnią się ilością danych, których używamy do obliczenia gradientu funkcji celu. W zależności od ilości danych dokonujemy kompromisu między dokładnością aktualizacji parametrów a czasem potrzebnym na wykonanie aktualizacji.²⁸

4.3. Algorytm propagacji wstecznej

Algorytm wstecznej propagacji jest nadzorowaną metodą uczenia się dla wielowarstwowych sieci sprzężenia zwrotnego z zakresu sztucznych sieci neuronowych. Zasada podejścia z propagacją wsteczną polega na modelowaniu danej funkcji przez modyfikację wewnętrznych wag sygnałów wejściowych w celu wytworzenia oczekiwanego sygnału wyjściowego. System jest szkolony przy użyciu nadzorowanej metody uczenia się, w której błąd między wyjściem systemu a znanym oczekiwanym wyjściem jest prezentowany systemowi i używany do modyfikowania jego stanu wewnętrznego. Z technicznego punktu widzenia algorytm wstecznej propagacji jest metodą uczenia wag w wielowarstwowej sieci neuronowej. W związku z tym wymaga zdefiniowania struktury sieci z co najmniej jednej warstwy, w której jedna warstwa jest w pełni połączona z

²⁷ Kiwiel, K., *Convergence and efficiency of subgradient methods for quasiconvex minimization*, Mathematical Programming, Series A, 2001

²⁸ Stachurski A., Wierzbicki A.: *Podstawy optymalizacji*, Oficyna Wydawnicza Politechniki Warszawskiej, 1999

następną. Standardowa struktura sieci to jedna warstwa wejściowa, jedna warstwa ukryta i jedna warstwa wyjściowa. Propagacja wsteczna może być używana zarówno w przypadku problemów związanych z klasyfikacją, jak i regresją. W problemach klasyfikacyjnych najlepsze wyniki uzyskuje się, gdy sieć ma jeden neuron w warstwie wyjściowej dla każdej wartości klasy.²⁹

²⁹ Rutkowski L., op. cit., s. 288-293.

Rozdział 5

Przedstawienie stworzonego algorytmu genetycznego oraz analiza wyników optymalizacji

5.1. Opis działania algorytmu genetycznego

Optymalizacja matematyczna zajmuje się problemem Algorytm genetyczny opracowany do celów pracy inżynierskiej opiera się na klasycznym schemacie algorytmów genetycznych. Jego celem jest określenie optymalnej ilości warstw oraz ilości neuronów w każdej z warstw sieci. Celem optymalizacji jest sieć neuronowa która będzie uczyć się sortować jeden z klasycznych zestawów danych – Iris flower data set.

Zmienne globalne możliwe do edycji w algorytmie:

- MIN_LAYER i MAX_LAYER – określa minimalną i maksymalną ilość warstw w sieci neuronowej
- MAX_NEURONS – określa maksymalną ilość neuronów na każdej z warstw sieci
- POPULATION – ilość osobników w populacji
- GENERATIONS – ile iteracji algorytmu maksymalnie zostanie wykonanych
- TARGET_SCORE – warunek zatrzymania
- MUTATION_CHANCE – szansa na mutację dla pojedynczego osobnika
- NO_EVOLVE_GENS – warunek zatrzymania poprzez brak dalszej ewolucji

A także dotyczącą sieci neuronowej zmienną STEPS opisującą ilość kroków podczas trenowania sieci neuronowej.

Pierwszym krokiem jest generacja początkowej populacji. Algorytm losuje ilość warstw oraz ilość neuronów w poszczególnych warstwach. Całość zapisywana jest jako jednowymiarowa tablica.

```
22 def generate_individual():
23     new_ind = []
24     layers = randint(MIN_LAYER, MAX_LAYER)
25     for n in range(0, layers):
26         new_ind.append(randint(1, MAX_NEURONS))
27     return new_ind
```

Rysunek 19 - Fragment kodu pokazujący tworzenie nowego osobnika

Źródło: opracowanie własne

Dla każdego z nich obliczana jest funkcja celu – dla celów testowania samego algorytmu używana jest zwykła funkcja działająca na tablicy jak na zbiorze liczb. W finalnej wersji liczenie kosztu jest skutecznością sortowania zbioru danych poprzez sieć neuronową o określonych parametrach. Jednowymiarowa tablica jest do tego celu zmieniana w dwuwymiarową tablicę w której pierwsza wartość to tabela zawierająca ilość warstw i neuronów w każdej z nich, a druga wartość to wartość funkcji celu, przykładowo:

[[3, 10, 1, 9, 4], 0.552]

Następnie algorytm sprawdza warunek stopu i w przypadku jego osiągnięcia kończy pracę. Jeśli warunek stopu nie został osiągnięty następują kolejne iteracje algorytmu w którym jest on poddawany selekcji oraz działaniu operatorów genetycznych: krzyżowania i mutacji.

Selekcja dokonywana jest dzięki symulacji ruletki. Tworzona jest tablica kosztów w której przechowywany jest relatywny koszt osobnika podzielony przez sumę wszystkich kosztów. Następnie przeliczana jest ona na sumę prawdopodobieństw dostania się do puli rodziców dla każdego z osobników. Na samym końcu tworzona jest nowa populacja, która jest pulą rodzicielską kolejnego pokolenia.

```

49 def roulette(pop_with_cost):
50     cost_sum = sum(n for _, n in pop_with_cost)
51
52     cost_table = []
53     for ind in pop_with_cost:
54         cost_table.append(ind[1])
55
56     rel_cost = [cost_ind / cost_sum for cost_ind in cost_table]
57     probability = [sum(rel_cost[:i + 1]) for i in range(len(rel_cost))]
58
59     new_pop = []
60     for n in range(POPULATION // 2):
61         r = random()
62         for (i, individual) in enumerate(pop_with_cost):
63             if r <= probability[i]:
64                 new_pop.append(individual)
65                 break
66     return new_pop

```

Rysunek 20 – Fragment kodu pokazujący symulację ruletki

Źródło: opracowanie własne

Następnie populacja zostaje poddana procesowi krzyżowania. Tworzona jest tablica kolejnych liczb naturalnych równa ilości osobników w puli rodzicielskiej, która będzie służyć jako indeks. Następnie za pomocą funkcji `shuffle()` tablica ulega losowej permutacji, a następnie według niej rodzice zostają po kolei dobrani w pary. Losowany jest punkt krzyżowania, po czym następuje krzyżowanie osobników i dodanie potomków do populacji.

```

69 def cross_parents(parent1, parent2, cross_point):
70     child = []
71     child.extend(parent1[:cross_point])
72     child.extend(parent2[cross_point:])
73     return child
74
75
76 def crossover(pop):
77     random_index_table = []
78
79     for x in range(0, len(pop)):
80         random_index_table.append(x)
81     shuffle(random_index_table)
82
83     for x in range(0, len(pop), 2):
84         p1 = pop[random_index_table[x]][0]
85         p2 = pop[random_index_table[x + 1]][0]
86         max_cross_point = min(len(p1), len(p2)) - 1
87         cross_point = randint(1, max_cross_point)
88         pop.append(cross_parents(p1, p2, cross_point))
89         pop.append(cross_parents(p2, p1, cross_point))
90     return pop

```

Rysunek 21 – Fragment kodu pokazujący proces krzyżowania

Źródło: opracowanie własne

Możliwe jest krzyżowanie osobników o różnych ilościach warstw poprzez warunek maksymalnego punktu przecięcia jako długości krótszej tabeli pomniejszonej o jeden, przykładowo:

$$A_1 = [1, 2 \mid 3]$$

$$A_2 = [6, 7 \mid 8, 9, 10, 11]$$

$$A'_1 = [1, 2 \mid 8, 9, 10, 11]$$

$$A'_2 = [6, 7 \mid 3]$$

W kolejnym kroku powstała populacja poddawana zostaje operacji mutacji. Mutacja ma określoną szansę i może dotyczyć zarówno rodzica jak i nowo powstałe poprzez krzyżowanie osobniki. Mutacja wybiera losową warstwę i zmienia ilość neuronów w tej warstwie o 1, z równym prawdopodobieństwem zwiększając ją lub zmniejszając. W przypadku gdy mutacja

zachodzi dla osobnika z puli rodzicielskiej z określonym już kosztem, koszt ten zostaje usunięty gdyż po zmianie parametrów sieci funkcja celu również ulegnie zmianie.

```
103 def mutate(pop):
104     mutated_pop = []
105     for ind in pop:
106         rand = random()
107         if rand <= MUTATION_CHANCE:
108             rand2 = randint(0, 100)
109             mut_value = 1 if rand2 % 2 == 0 else -1
110             if isinstance(ind[0], list): # individual has cost:
111                 target_cell = randint(0, len(ind[0]) - 1)
112                 new_value = ind[0][target_cell] + mut_value
113                 new_value = 1 if new_value > MAX_NEURONS else new_value
114                 new_value = MAX_NEURONS if new_value == 0 else new_value
115                 ind[0][target_cell] = new_value
116                 mutated_pop.append(ind[0]) # so the cost function will be cleared
117             else:
118                 target_cell = randint(0, len(ind) - 1)
119                 new_value = ind[target_cell] + mut_value
120                 new_value = 1 if new_value > MAX_NEURONS else new_value
121                 new_value = MAX_NEURONS if new_value == 0 else new_value
122                 ind[target_cell] = new_value
123                 mutated_pop.append(ind)
124         else:
125             mutated_pop.append(ind)
126     return mutated_pop
```

Rysunek 22 – Fragment kodu pokazujący proces mutacji

Źródło: opracowanie własne

Na końcu obliczana jest wartość kosztu dla nowo powstałych lub zmutowanych osobników, wybierany najwyższy wynik w tej generacji, po czym algorytm wykonuje kolejną iterację zaczynając od sprawdzenia czy warunek stopu został osiągnięty.

Algorytm posiada trzy sposoby na zakończenie pracy: osiągnięcie warunku zatrzymania, osiągnięcie określonej wartości iteracji (generacji) oraz warunek braku dalszej ewolucji.

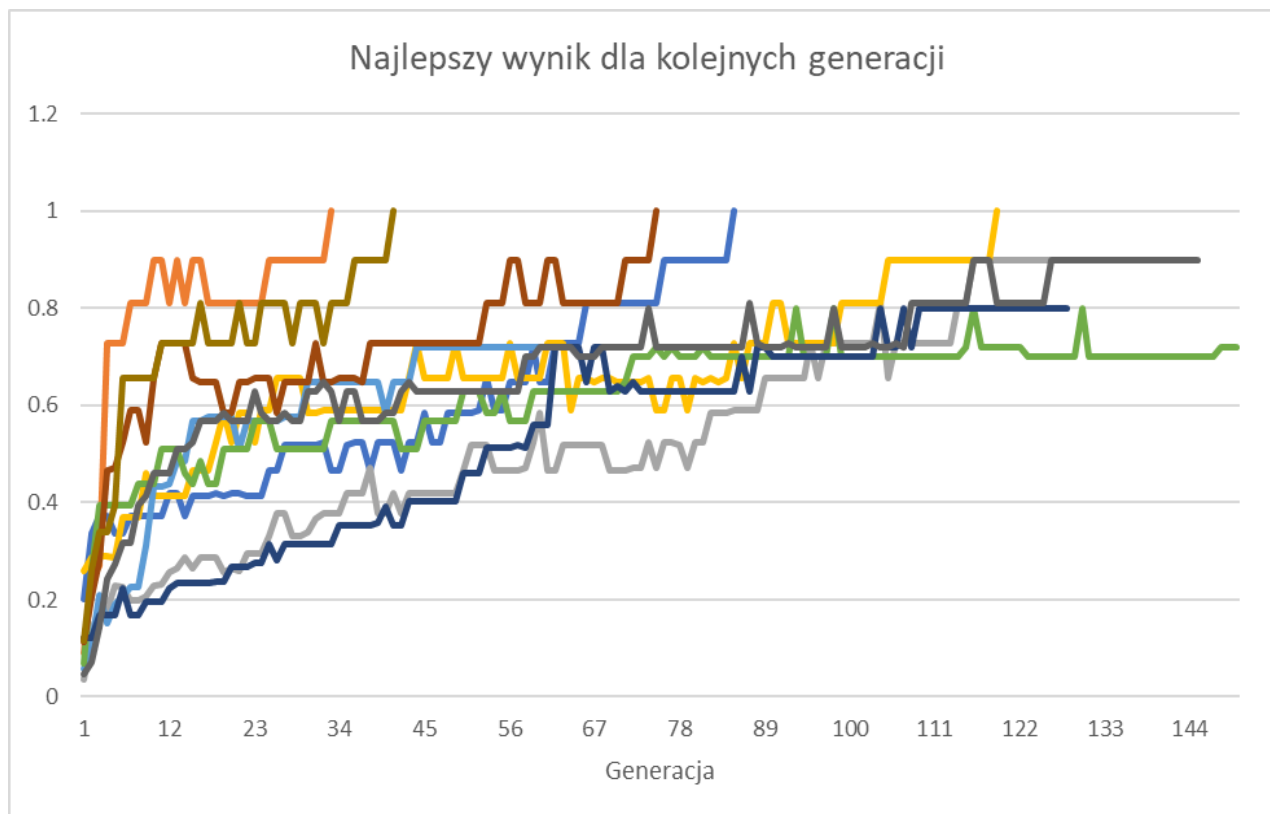
5.2. Optymalizowana sieć neuronowa

Zestaw danych Iris zawiera cztery cechy (długość i szerokość działek i płatków) z 50 próbek trzech gatunków Iris (Iris setosa, Iris virginica i Iris versicolor). Miary te posłużyły do stworzenia liniowego modelu dyskryminacyjnego do klasyfikacji gatunków. Zbiór danych jest

często używany w przykładach eksploracji danych, klasyfikacji i grupowania oraz do testowania algorytmów. W oparciu o liniowy model dyskryminacyjny ten zbiór danych stał się typowym przypadkiem testowym dla wielu technik klasyfikacji statystycznej w uczeniu maszynowym, takich jak maszyny wektorów nośnych czy głębokie sieci neuronowe. To sprawia, że zbiór danych jest dobrym przykładem do wyjaśnienia różnicy między technikami nadzorowanymi i nienadzorowanymi w eksploracji danych. Liniowa Analiza Dyskryminacyjna należy do metod statystycznych, dokonuje podziału na obszary graniczne funkcjami liniowymi, spisuje się dobrze na prostych w analizie danych. Jest jedną z metod klasyfikacji danych. Zadaniem takich metod jest określenie przynależności do klas innych badanych obiektów opisanych tymi samymi cechami.

5.3. Wyniki optymalizacji

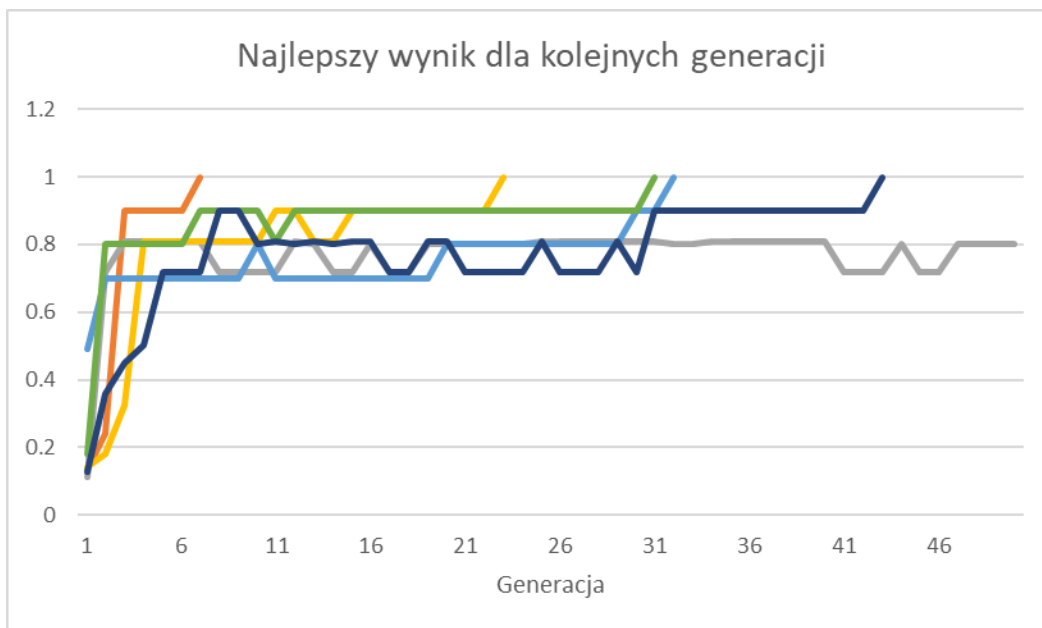
Do sprawdzenia działania samego algorytmu genetycznego tablica z ilością warstw i neuronów była traktowana jak zwykła tablica liczb, która zostawała poddawana operacjom arytmetycznym. Wybrane zostały do tego różne równania w celu sprawdzenia jak algorytm radzi sobie z różnym stopniem skomplikowania. Poniżej przedstawione są przykładowe wyniki dla trzech różnych funkcji i wartości.



Rysunek 23 – Wykres najlepszych wyników dla kolejnych generacji

Źródło: opracowanie własne

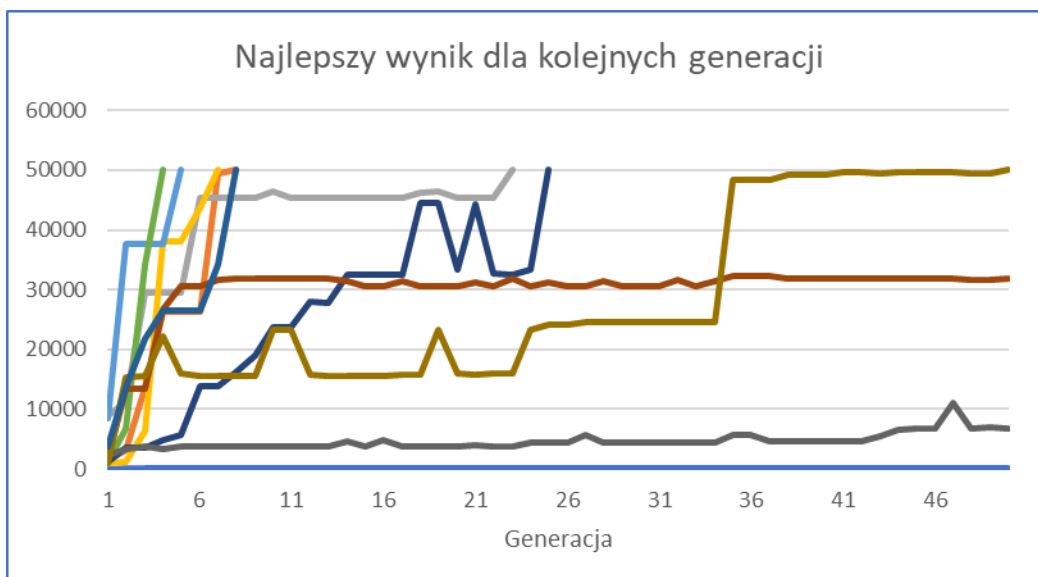
- Funkcja kosztu: $c = i[0] * i[1] * i[2] * i[3] / (i[4] * i[5] * 100)$
- Ilość warstw: 6
- Maksymalna ilość neuronów: 10



Rysunek 24 – Wykres najlepszych wyników dla kolejnych generacji

Źródło: opracowanie własne

- Funkcja kosztu: $c = i[1] * i[2] * i[3] / 1000$
- Ilość warstw: 3
- Maksymalna ilość neuronów: 10



Rysunek 25 – Wykres najlepszych wyników dla kolejnych generacji

Źródło: opracowanie własne

- Funkcja kosztu: $c = (i[0] * i[2] * i[4] * i[6] - (i[1] * i[3] * i[5])) / (i[7] * i[3] * i[0])$
- Ilość warstw: 8
- Maksymalna ilość neuronów: 50

Każda z linii na wykresach to osobny algorytm genetyczny uruchomiony z takimi samymi warunkami początkowymi. Niektóre z nich osiągały wyniki bardzo szybko, inne wolniej., jednak wykazano skuteczność algorytmu. Częstym zjawiskiem było zatrzymanie się poziomu na określonym poziomie i brak postępów aż do czasu zajścia pozytywnej mutacji – wynika to z właściwości algorytmów genetycznych.

Po ustalenia funkcji kosztu jako wyniku odgadywania zbioru danych typu Iris przez sieć neuronową wyniki zachowują podobną tendencję wzrostową. Poniżej opisany został wynik optymalizacji sieci neuronowej o podanych parametrach przez algorytm genetyczny:

- Minimalna ilość warstw:4
- Maksymalna ilość warstw: 6
- Maksymalna ilość neuronów: 8
- Populacja: 80
- Generacje: 30
- Warunek stopu: 0.995
- Szansa mutacji: 0.05
- Warunek braku ewolucji: 6



Rysunek 26 – Wykres najlepszych wyników optymalizowania sieci neuronowej

Źródło: opracowanie własne

W tym przykładzie w 16 generacji nastąpił brak dalszych ewolucji i wynik ustabilizował się na poziomie 96.7% skuteczności, po czym po sześciu generacjach algorytm został zatrzymany.

Ciekawym zjawiskiem jakie występuje przy optymalizacji sieci neuronowych do działania na tym zbiorze danych jest fakt, że sieć neuronowa o tym samym schemacie może mieć zupełnie inne wyniki. W poniższej tabeli zamieszczone są wyniki pięciu najlepszych schematów sieci neuronowej w ostatniej generacji algorytmu.

	Schemat sieci	Wynik	Ilość w populacji	Średni wynik
1	[6, 6, 5, 6]	0.966666639	8	0.600000024
2	[6, 6, 5, 6]	0.966666639	8	0.600000024
2	[6, 6, 6, 6]	0.933333337	8	0.587499976
3	[6, 4, 8, 6, 5]	0.899999976	3	0.766666651
4	[6, 3, 5, 6]	0.899999976	1	0.899999976

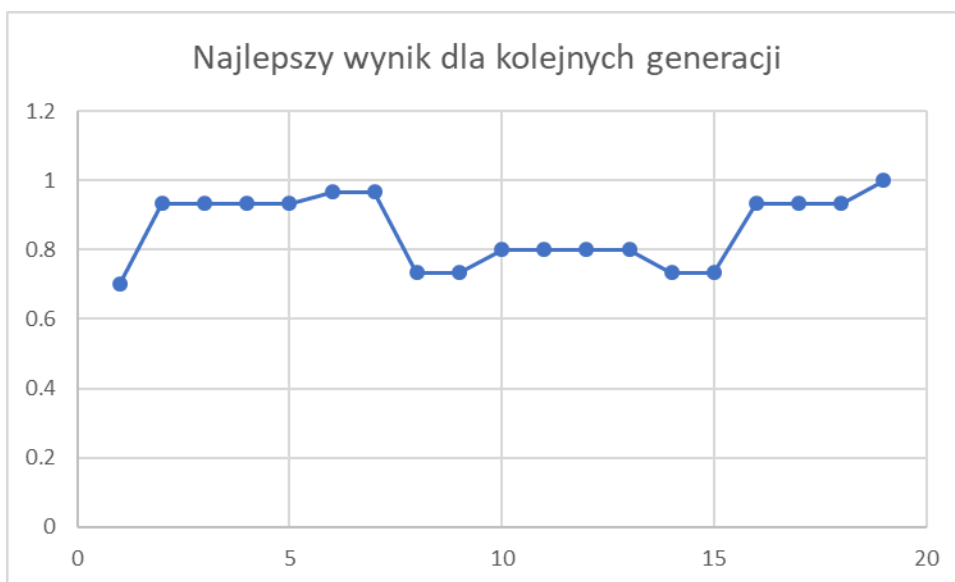
Tabela 3 – Najlepsze wyniki oraz częstotliwość występowania w populacji

Źródło: Opracowanie własne

Schemat sieci [6, 6, 5, 6] posiada najlepszą skuteczność. Jednak średni wynik dla tej sieci to 60% skuteczności, a najgorszy z nich to 23%. Przez taką właściwość optymalizowanej sieci neuronowej proces krzyżowania i mutacji jest mocno osłabiony. Mimo tego osobniki z wysoką średnią skutecznością mają dużo większą szansę na rozmnożenie się, więc podstawowe założenia

algorytmu są spełniane. W tym przykładzie dwa rodzaje sieci neuronowej mające najwyższą skuteczność – [6, 6, 5, 6] oraz [6, 6, 6, 6] stanowią aż 20% całej populacji.

Poniżej przedstawiony jest wykres najlepszych wyników po zmianie ilości warstw na przedział (2, 4) oraz zmniejszeniu populacji do 32:



Rysunek 27 – Wykres najlepszych wyników optymalizowania sieci neuronowej

Źródło: Opracowanie własne

Algorytm poprzez małą wielkość populacji jest bardziej podatny na mutacje i krzyżowanie, w tym przypadku po początkowym wzroście do 97% skuteczności w generacji szóstej nastąpił spadek, po czym algorytm wygenerował lepszy wynik i osiągnął cel dopiero w generacji 19.

W przypadku znacznego zwiększania populacji, lub ustalania większych wartości dla maksymalnej ilości warstw czy neuronów algorytm osiągał docelową skuteczność bardzo szybko. Jest to spowodowane prostotą modelu Iris, który przy bardziej rozbudowanej sieci neuronowej osiąga wysoką skuteczność prawie zawsze. Przykładowo dla ilości warstw w zakresie (4, 8), maksymalnej ilości neuronów równej 20 oraz populacji równej 120 algorytm osiągnął cel 100% w trzeciej generacji, przy czym ponad połowa wygenerowanych sieci miała skuteczność większą niż 95%.

Zakończenie

Celem pracy było ukazanie pozytywnych aspektów optymalizacji technologii sztucznej inteligencji za pomocą algorytmów genetycznych, w celu ustalenia optymalnej topologii sztucznej sieci neuronowej. W powyższym przykładzie został zbadany tylko jeden rodzaj sieci neuronowej oraz jeden model algorytmu genetycznego. W warunkach przyjętych w powyższej pracy udało się wskazać że jest możliwe osiągnięcie wyższej skuteczności w klasyfikowaniu obrazów za pomocą algorytmów genetycznych. W zależności od założonych celów implementacja tego algorytmu na istniejących już sieciach neuronowych może być opłacalna i pożądana.

Osobnym zagadnieniem pozostaje kwestia opłacalności takiego rozwiązania ze względu na długość działania algorytmu bądź zużycia zasobów w postaci mocy obliczeniowej maszyn na których algorytm miałby działać. Również niezależnym zagadnieniem wymagającym dodatkowych badań jest istotność posiadania tak wysokiej skuteczności w komercyjnych zastosowaniach sztucznej inteligencji. Problem jest złożony i wielowymiarowy i w najbliższych latach możemy oczekiwać większej ilości badań poruszających to zagadnienie.

Bibliografia

Opracowania książkowe:

1. Goldberg D. E., Algorytmy genetyczne i ich zastosowania, Wydawnictwo Naukowo-Techniczne, Warszawa 2003
2. Górską T., Grabowska A., Zagrodzka J., *Mózg a zachowanie*. Wydawnictwo Naukowe PWN, Warszawa, 2012
3. Hastie T., Tibshirani R., Friedman J., *The Elements of Statistical Learning: Data mining, Inference, and Prediction*, Wydawnictwo Springer, New York 2009
4. Herz J., Krogh A., Palmer R.G., *Wstęp do teorii obliczeń neuronowych*, Wydawnictwa Naukowo-Techniczne, Warszawa 1995.
5. Hinton G.; Sejnowski, T., *Unsupervised Learning: Foundations of Neural Computation*, MIT Press, 1999
6. Kosiński R. A., *Sztuczne sieci neuronowe. Dynamika nieliniowa i chaos*, Wydawnictwo Naukowe PWN SA, Warszawa 2018
7. Kossut M., *Mechanizmy plastyczności mózgu*. Wydawnictwo Naukowe PWN, Warszawa 1994
8. Krause M., *Człowiek i jego układ nerwowy*, Wydawnictwo Śląsk, Katowice 2003
9. Longstaff A., *Neurobiologia*. Wydawnictwo Naukowe PWN, Warszawa 2002
10. McIlwraith D., Marmanis H., Babenko D., *Inteligentna sieć. Algorytmy przyszłości.*, Wydawnictwo Helion, Gliwice 2017
11. Mirjalili V., Sawka K., *Python: uczenie maszynowe*, Wydawnictwo Helion, Gliwice 2019
12. Osowski S., *Sieci neuronowe do przetwarzania informacji*. Wydawnictwo OW PW, Warszawa 2013
13. Rutkowska D., Piliński M., Rutkowski L., *Sieci neuronowe, algorytmy genetyczne i systemy rozmyte*, Wydawnictwo Naukowe PWN, Warszawa 1997.
14. Rutkowski L., *Metody i techniki sztucznej inteligencji*, Wydawnictwo naukowe PWN, Warszawa 2012

15. Stachurski A., Wierzbicki A.: *Podstawy optymalizacji*, Oficyna Wydawnicza Politechniki Warszawskiej, 1999
16. Tadeusiewicz R., Kot L., Mikrut Z., Majewski J., *Biocybernetyka cz. I*, Wydawnictwo AGH, Kraków, 1982.
17. Tadeusiewicz R., Szaleniec M., *Leksykon sieci neuronowych*, Wydawnictwo Fundacji „Projekt Nauka”, Wrocław
18. Zieliński T., *Cyfrowe przetwarzanie sygnałów od teorii do zastosowań*. Wydawnictwa komunikacji i łączności, Warszawa 2007

Artykuły naukowe:

1. Cybenko, G., *Approximation by superpositions of a sigmoidal function* *Mathematics of Control, Signals, and Systems*, 1989.
2. Goodfellow I., Bengio Y., Courville A., *Deep learning*. MIT Press, 2016.
3. Grabowski M., *Zastosowanie samoorganizujących się map cech Kohonena w analizie danych*, 1997
4. Griewank, A., *Who Invented the Reverse Mode of Differentiation?* 2012
5. Kasprzak, W., *Rozpoznawanie obrazów i sygnałów mowy*, Warszawa 2009
6. Kiwiel, K., *Convergence and efficiency of subgradient methods for quasiconvex minimization*, *Mathematical Programming, Series A*, 2001
7. LeCun Y., Bengio Y., Hinton G., *Deep learning*, *Nature*, 2015.
8. McCulloch W., Pitts W., *A logical calculus of the ideas immanent in nervous activity*. In: *Bulletin of Mathematical Biophysics*, Bd. 5, 1943
9. Nair V., Hinton G.E., *Rectified linear units improve restricted boltzmann machines*, Haifa, 2010
10. Rostamizadeh A., Talwalkar A. (2012) *Foundations of Machine Learning*, The MIT Press, 2012
11. Turian J., Bergstra J., Bengio Y., *Quadratic features and deep architectures for chunking*, in *Proceedings of Human Language Technologies: The 2009 Annual Conference of the North American Chapter of the Association for Computational Linguistics*, vol. Companion Volume, 2009

Źródła internetowe:

1. <http://www.dkriesel.com> - David Kriesel, 2007, A Brief Introduction to Neural Networks, dostęp na 16.01.2021

Spis rysunków

RYSUNEK 1 - SCHEMAT BLOKOWY ALGORYTMU GENETYCZNEGO	9
RYSUNEK 2 - TARCZA RULETKI	11
RYSUNEK 3 - SCHEMAT NEURONU	15
RYSUNEK 4 - ZDJĘCIE NEURONU WYKONANE MIKROSKOPEM ELEKTRONOWYM	16
RYSUNEK 5 - SCHEMAT PROJEKTU PERCEPTRONU	18
RYSUNEK 6 - SCHEMAT PERCEPTRONU	20
RYSUNEK 7 - ZALEŻNOŚĆ ZŁOŻONOŚCI PROBLEMU WZGLĘD NIEZNAJOMOŚCI REGUŁ	23
RYSUNEK 8 - FUNKCJA SIGMOIDALNA	25
RYSUNEK 9 - TANGENS HIPERBOLICZNY	26
RYSUNEK 10 - FUNKCJA SOFTMAX	27
RYSUNEK 11 - FUNKCJA SOFTSIGN	28
RYSUNEK 12 - FUNKCJA RELU	29
RYSUNEK 13 - FUNKCJA ELU	31
RYSUNEK 14 - RADIALNA FUNKCJA BAZOWA NA POCZĄTKU ANALIZY DANYCH	33
RYSUNEK 15 - RADIALNA FUNKCJA BAZOWA PO DOKONANIU ORGANIZACJI DANYCH	33
RYSUNEK 16 - SCHEMAT BLOKOWY SAMOORGANIZUJĄCEJ SIĘ SIECI NEURONOWEJ KOHONENA	34
RYSUNEK 17 - SCHEMAT REKURENCYJNEJ SIECI NEURONOWEJ	35
RYSUNEK 18 - ILOCZYN SKALARNY WEKTORÓW	36
RYSUNEK 19 - FRAGMENT KODU POKAZUJĄCY TWORZENIE NOWEGO OSOBNIKA	45
RYSUNEK 20 – FRAGMENT KODU POKAZUJĄCY SYMULACJE RULETKI	46
RYSUNEK 21 – FRAGMENT KODU POKAZUJĄCY PROCES KRZYŻOWANIA	47
RYSUNEK 22 – FRAGMENT KODU POKAZUJĄCY PROCES MUTACJI	48
RYSUNEK 23 – WYKRES NAJLEPSZYCH WYNIKÓW DLA KOLEJNYCH GENERACJI	50
RYSUNEK 24 – WYKRES NAJLEPSZYCH WYNIKÓW DLA KOLEJNYCH GENERACJI	51
RYSUNEK 25 – WYKRES NAJLEPSZYCH WYNIKÓW DLA KOLEJNYCH GENERACJI	51
RYSUNEK 26 – WYKRES NAJLEPSZYCH WYNIKÓW OPTYMALIZOWANIA SIECI NEURONOWEJ	53
RYSUNEK 27 – WYKRES NAJLEPSZYCH WYNIKÓW OPTYMALIZOWANIA SIECI NEURONOWEJ	54

Spis tabel

TABELA 1 - WARTOŚCI PRZYSTOSOWANIA DLA PRZYKŁADOWYCH ORGANIZMÓW	10
TABELA 2 - WARTOŚCI GENÓW W CHROMOSOMIE PRZED I PO MUTACJI	13
TABELA 3 – NAJLEPSZE WYNIKI ORAZ CZĘSTOTLIWOŚĆ WYSTĘPOWANIA W POPULACJI	53